Tiny-YOLO from scratch to an FPGA: a bit-exact chain from NumPy to an HLS accelerator driven by an ARM SoC

Léonard Rivals

Draft, rev. `9fd9562-dirty`, 2026-10-22

Abstract

We rewrite the Tiny-YOLOv2 and Tiny-YOLOv3 detectors from scratch in NumPy, without any deep-learning library, with forward and backward passes checked by finite differences. With the published Darknet weights, the float model reaches 56.30 mAP on VOC2007 test (4,952 images). We fuse batch normalization, quantize weights per channel and activations per layer to INT8, and run the network in integer arithmetic only: the gap to the float model is −0.64 point (55.66 mAP), without quantization-aware training. A C++ golden model and a single-engine Vitis HLS accelerator reproduce the integer model byte for byte; driven by the ARM driver over the C-simulated kernel, the whole test set gives $4{,}952^{\text{C-sim}}$ identical images, hence $55.66^{\text{C-sim}}$ mAP. On a Kria KV260 at 200 MHz, the cycle model of the optimized kernel projects $37.2^{\text{proj.}}$ ms per image, i.e. $187.3^{\text{proj.}}$ GOPS for 3.49 GMAC; the board itself has not been measured yet. Extensions compare power-of-two weights, 4-bit quantization, integer hardware post-processing (55.56 mAP) and a streaming architecture ($32.1^{\text{est.}}$ images/s); packing two INT8 multiplications per DSP brings the projection to $24.7^{\text{proj.}}$ ms with the same DSP count. The same chain runs YOLOv4-tiny on COCO (41.4 AP50 in float, −1.3 point in INT8), and YOLOv7-tiny is being brought onto it. A dependability layer injects bit flips into the integer model and the kernel buffers, checks every convolution with an exact int32 checksum (ABFT) down to the HLS kernel, and flags out-of-distribution inputs from the backbone features (AUROC 0.88 on thermal images). Safety evidence follows: an equivalence matrix per layer and stage, with the integer model, the golden model and the kernel equal byte for byte and 95 SMT obligations proved on the scalar kernels, and a static latency bound that does not depend on the image ($92.4^{\text{proj.}}$ ms per frame for Tiny-YOLOv2). First sweeps fine-tune Tiny-YOLOv3 on five other datasets.

Status of the numbers. A superscript marks a number that is not a measurement on the full test set: $^{\text{C-sim}}$ computed by the HLS kernel in C simulation, $^{\text{proj.}}$ cycle or resource model without synthesis, $^{\text{est.}}$ analytical estimate, $^{\text{tier R}}$ subset of images (M12 rule), $^{\text{pub.}}$ figure taken from a paper. Figure captions come from the figure registry of the repository and are still in French until milestone M19 translates them.

1 Introduction and contributions

Object detection on embedded FPGAs is usually reached through vendor toolchains: a network trained in a deep-learning framework is quantized by a vendor tool and mapped to a preconfigured processing unit, as with the Vitis AI DPU used by Fata and Elmannai [1] on the same board as ours. Every layer of that stack hides choices that change the result: preprocessing, rounding, the integer approximation of the activation, the head decoding. This work takes the opposite route and rebuilds each layer by hand, so that every number can be traced to a line of code and checked against the stage before it.

The YOLO family predicts boxes on a grid in a single pass over the image [2, #002.0]; Tiny-YOLOv2 keeps one 13×13 output map [3, #002.2] and Tiny-YOLOv3 adds a second one at 26×26 [4, #004.1]. These two networks are small enough for a mid-range Zynq UltraScale+ and are the

reference workload of most FPGA papers of the base (Section 2). YOLOv4-tiny and YOLOv7-tiny, more accurate on COCO at a similar cost, keep the leaky activation and the Darknet format, and test how far the same engine goes (outside the base).

Contributions:

1. **A complete chain without a learning library**: layers, losses, training, decoding, NMS and VOC evaluation in NumPy, each gradient checked by finite differences (Sections 3 and 4).
2. **Bit-exactness from the NumPy integer model to the HLS kernel**: the integer model, the C++ golden model and the C-simulated kernel behind the ARM driver produce the same bytes, verified on the whole VOC2007 test set (Sections 5, 6 and 8).
3. **One engine for v2, v3 and YOLOv4-tiny**: a single parameterized layer-by-layer accelerator runs the three networks from a manifest, with route, grouped route and upsample handled by addressing (Section 7).
4. **Formats compared on the same model**: INT8, power-of-two weights after REQ-YOLO, 4-bit weights and activations, and integer post-processing, all evaluated on the same test set with the same protocol (Section 9).
5. **Dependability checks on the integer model**: fault injection, an exact ABFT checksum per convolution in Python, in the golden model and in the HLS kernel, and an out-of-distribution monitor that costs a negligible share of the detector (Section 10).
6. **Safety evidence**: a model ↔ circuit equivalence matrix per layer and stage, exhaustive operator tests, an SMT proof of the scalar kernels, traceability checked in CI, and a static latency bound of the accelerator (Section 11).
7. **Reproducible tooling**: tiers of test profiles, figures, notebooks and this article are regenerated from the repository by `make` targets (Section 14).

State of the project at the last render: 231 of 397 tasks done across 41 milestones, 7 milestones complete (details in Section 13).

2 Related work

Two families of CNN accelerators coexist on FPGAs [5, #004.0]. A *streaming* architecture instantiates one hardware stage per layer and pipelines them, which gives latency and throughput but needs one bitstream per network; SATAY keeps all parameters on chip this way [6, #006.0]. A *single engine* runs the layers one after the other on a systolic array or a matrix unit, scheduled by software [5, #007.1]; it fits a small Zynq and serves several networks with one bitstream, at the price of DDR traffic at every layer. The tiling of the convolution loop nest and its roofline analysis come from Zhang et al. [7, #009.0, #010.2].

Table 1 lists the results of the base (specification §10.4) next to ours. Most rows cannot be compared directly with this work, for reasons the numbers alone do not show:

- **2024-zhang** is the only reference on the same model (Tiny-YOLOv2 416, INT8, batch of 1). It runs on an FPGA without an ARM core, packs two multiplications per DSP [8, #011.0] and includes hardware post-processing [8, #013.1].
- **2026-fata** runs on the same KV260, but with a preconfigured DPU, a YOLOv3-tiny pruned at 70 % and QAT [1, #015.0]: pruning divides the MACs by about three, so images per second do not convert.
- **2023-zhai** uses INT16 and a pruned network on a vehicle dataset; only the order of magnitude of its GOPS compares [4, #004.1].
- **2019-ding** (REQ-YOLO) uses FFT-based convolutions and power-of-two weights chosen by ADMM [9, #008.1, #009.4], on another architecture.
- **2025-kim** is a slow functional prototype of the full YOLOv2 on a Zybo, with decoding on the ARM [10, #026.0]: not a performance reference.
- Power is reported with different perimeters (chip estimate, board, unspecified); ours will be

measured both on the chip (`report_power`) and on the SOM (INA260).

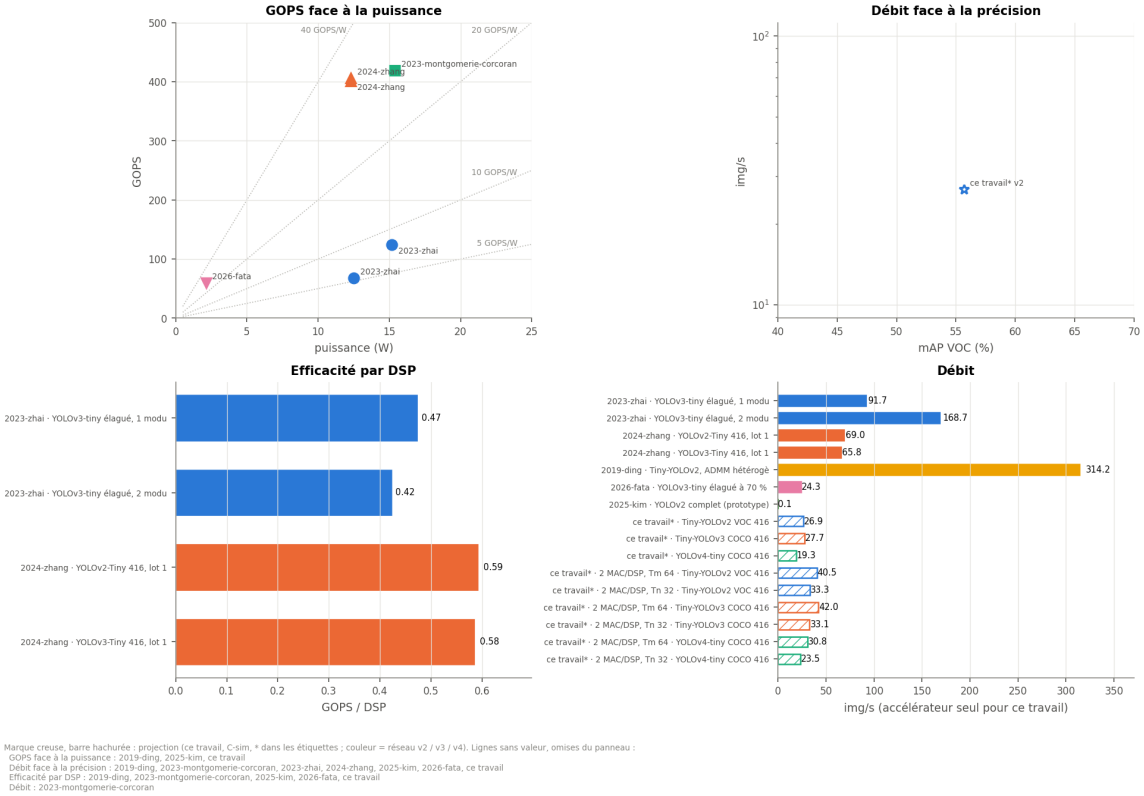


Figure 1: Comparaison aux accélérateurs publiés : GOPS et puissance, débit et mAP, GOPS par DSP, débit (marque creuse : projection de ce travail, par réseau et variante M23). (`etat_art`)

3 Networks and NumPy building blocks

Tiny-YOLOv2 is a stack of nine 3×3 convolutions with batch normalization and leaky ReLU, six max-pooling layers (the last one 2×2 with stride 1) and a linear 1×1 head with $5 \times (5 + C)$ filters; Tiny-YOLOv3 adds a second head at 26×26 fed by an upsample and a route that concatenates an earlier map [11, #005.0], [4, #016.2]. Counted by `make count-macs` from the network descriptions, Tiny-YOLOv2 VOC has 15.86 M parameters and 3.49 GMAC at 416×416, in agreement with the count of CoDeNet [12, #025.0]; Tiny-YOLOv3 VOC has 8.71 M parameters and 2.74 GMAC (Figures 2 and 3).

YOLOv4-tiny (`tiny-yolov4-coco`, outside the base) replaces the plain backbone by CSP blocks: a *grouped route* takes the second half of the channels of a map, and the heads rescale the box centres by `scale_x_y`. It has 6.06 M parameters and 3.45 GMAC (Figure 4). The grouped route is one more addressing mode of the engine: it reads a channel range of an existing buffer, so it costs no compute and no copy. YOLOv7-tiny (`tiny-yolov7-coco`, 6.23 M parameters, 2.89 GMAC) adds ELAN blocks, whose concatenations have four sources, and a spatial pyramid pooling with stride-1 max pools of 5×5, 9×9 and 13×13. The export rewrites the two larger pools as chained 5×5 pools and copies the pooled input by an identity 1×1 convolution; this network is work in progress (float, calibration and export done, accuracy not yet measured). YOLOv5n and YOLOv8n (Ultralytics, SiLU activation, anchor-free head for v8n, outside the base) are read from their checkpoints without PyTorch: a reader of the pickle format and the network rebuilt from its YAML description give a trainable NumPy YOLOv5n equal to the reference to about 10^{-14} (M33).

Every layer exposes the same interface, `forward(x) -> (y, cache)` and `backward(dy, cache) -> (dx, grads)`. The convolution exists in a naive version, kept as the test reference, and an im2col

Table 1: Published results (`results/benchmarks.csv`) and this work.

Work	Model	FPGA	Format	img/s	ms	GOPS	W	Status
2023-zhai	YOLOv3-tiny pruned, 1 module	Zynq XC7Z035	INT16	91.65	—	67.91	12.51	published
2023-zhai	YOLOv3-tiny pruned, 2 modules	Zynq XC7Z035	INT16	168.72	—	124.01	15.18	published
2024-zhang	YOLOv2-Tiny 416, batch 1	Kintex-7 325T	INT8	—	14.49	406	12.3	published
2024-zhang	YOLOv3-Tiny 416, batch 1	Kintex-7 325T	INT8	—	15.2	401	12.3	published
2023-montgomerie-corcoran	YOLOv3-Tiny 416	VCU110	W8A16	—	—	418.9	15.4	published
2019-ding	Tiny-YOLOv2, ADMM heterogeneous	Virtex-7 690T	FFT + powers of 2	314.2	—	—	21	published
2026-fata	YOLOv3-tiny pruned at 70 % (DPU Vitis AI)	Kria KV260 (DPU)	INT8 QAT	24.3	—	60.18	2.13	published
2025-kim	YOLOv2 full (prototype)	Zybo Z7-20	INT16	0.083	12,000	—	—	published
this work (C-sim projection)	Tiny-YOLOv2 VOC 416	Kria KV260 (XCK26)	INT8 (W8A8, per channel)	26.87	37.2	187.3	—	projection
this work (C-sim projection)	Tiny-YOLOv3 COCO 416	Kria KV260 (XCK26)	INT8 (W8A8, per channel)	27.72	36.1	154.3	—	projection
this work (C-sim projection)	YOLOv4-tiny COCO 416	Kria KV260 (XCK26)	INT8 (W8A8, per channel)	19.30	51.8	133.3	—	projection
this work (C-sim projection, 2 MACs/DSP, Tm 64)	Tiny-YOLOv2 VOC 416	Kria KV260 (XCK26)	INT8 (W8A8, per channel)	40.53	24.7	282.5	—	projection
this work (C-sim projection, 2 MACs/DSP, Tn 32)	Tiny-YOLOv2 VOC 416	Kria KV260 (XCK26)	INT8 (W8A8, per channel)	33.31	30.0	232.2	—	projection
this work (C-sim projection, 2 MACs/DSP, Tm 64)	Tiny-YOLOv3 COCO 416	Kria KV260 (XCK26)	INT8 (W8A8, per channel)	41.95	23.8	233.4	—	projection
this work (C-sim projection, 2 MACs/DSP, Tn 32)	Tiny-YOLOv3 COCO 416	Kria KV260 (XCK26)	INT8 (W8A8, per channel)	33.06	30.2	184.0	—	projection
this work (C-sim projection, 2 MACs/DSP, Tm 64)	YOLOv4-tiny COCO 416	Kria KV260 (XCK26)	INT8 (W8A8, per channel)	30.79	32.5	212.7	—	projection
this work (C-sim projection, 2 MACs/DSP, Tn 32)	YOLOv4-tiny COCO 416	Kria KV260 (XCK26)	INT8 (W8A8, per channel)	23.51	42.5	162.4	—	projection

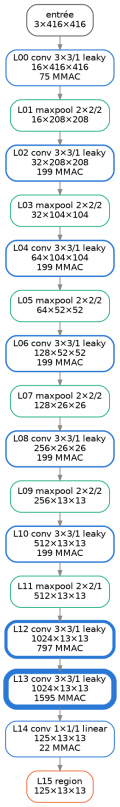


Figure 2: Graphe couche par couche (type, noyau et stride, forme de sortie C×H×W ; bord épais = beaucoup de MACs). (`graphe`)

Tiny-YOLOv2 VOC — fiche couche par couche (yolo.models.specs : infer_shapes, layer_cost)

L	type	k/s/pad	cin → cout	entrée	sortie	paramètres	MACs (M)	part des MACs	champ r	pas j	act.	BN
00	conv	3/1/1	3 → 16	416×416	416×416	464	74.8	2.1 %	3	1	leaky	oui
01	maxpool	2/2	16 → 16	416×416	208×208				4	2		
02	conv	3/1/1	16 → 32	208×208	208×208	4 672	199.4	5.7 %	8	2	leaky	oui
03	maxpool	2/2	32 → 32	208×208	104×104				10	4		
04	conv	3/1/1	32 → 64	104×104	104×104	18 560	199.4	5.7 %	18	4	leaky	oui
05	maxpool	2/2	64 → 64	104×104	52×52				22	8		
06	conv	3/1/1	64 → 128	52×52	52×52	73 984	199.4	5.7 %	38	8	leaky	oui
07	maxpool	2/2	128 → 128	52×52	26×26				46	16		
08	conv	3/1/1	128 → 256	26×26	26×26	295 424	199.4	5.7 %	78	16	leaky	oui
09	maxpool	2/2	256 → 256	26×26	13×13				94	32		
10	conv	3/1/1	256 → 512	13×13	13×13	1 180 672	199.4	5.7 %	158	32	leaky	oui
11	maxpool	2/1	512 → 512	13×13	13×13				190	32		
12	conv	3/1/1	512 → 1024	13×13	13×13	4 720 640	797.4	22.9 %	254	32	leaky	oui
13	conv	3/1/1	1024 → 1024	13×13	13×13	9 439 232	1594.9	45.8 %	318	32	leaky	oui
14	conv	1/1/0	1024 → 125	13×13	13×13	128 125	21.6	0.6 %	318	32	linear	non
15	region		125 → 125	13×13	13×13				318	32		
total						**15 861 773**	**3.486 G**					

Figure 3: Fiche couche par couche : type, noyau, formes, paramètres, MACs et leur part, champ réceptif et pas cumulé (CSV à côté de l'image). (`fiche`)

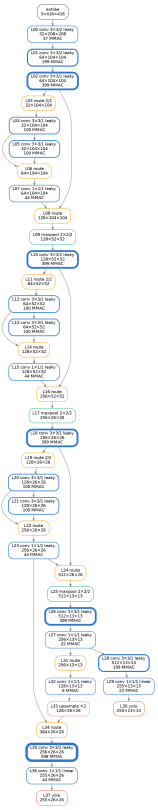


Figure 4: Graphe couche par couche (type, noyau et stride, forme de sortie C×H×W ; bord épais = beaucoup de MACs). (`graphe`)

version used for training; both agree to 10^{-12} (Figure 5). The weight gradient is the correlation of the output gradient with the input [13, #013.0]. Each layer passes a gradient check with centered differences in float64, using the max norm over the tensor so that a gradient that happens to be small is not drowned in rounding noise (outside the base; Figure 6).

The loss is the YOLOv3 one: direct regression of the box offsets, whose gradient is "ground truth minus prediction" [11, #003.0], logistic objectness with an *ignore* rule for predictions that overlap a ground truth without being responsible for it [11, #003.1], and multi-label binary cross-entropy for the classes [11, #004.0]. Targets are assigned to the best anchor by IoU of the shapes; anchors come from k-means with the distance $1-\mathrm{IoU}$ [3, #002.4]. The training loop (SGD, burn-in, multi-scale inputs [3, #003.2]) overfits a single image, which validates targets, loss and optimizer together (Figure 7).

4 Inference and evaluation

Inference decodes each head with the YOLOv2 parameterization [3, #003.1], filters predictions by confidence, then applies a per-class NMS at IoU 0.45. Evaluation follows the VOC2007 protocol: 11-point AP, `difficult` objects ignored, confidence threshold 0.005; our implementation matches the official devkit to 10^{-12} (`python/tests/test_metrics.py`).

With the published Darknet weights the reference mAP is 56.30, against $57.1^{\text{pub.}}$ published by Darknet. Preprocessing explains most of what separates the variants (Table 2): these weights were trained by the Darknet of the YOLOv2 era, which resizes the image to a square without keeping its aspect ratio. A letterbox, as in our training pipeline, shrinks and distorts the objects differently than in training and loses about two points; reproducing the Darknet bilinear interpolation instead of Pillow's barely changes the result. The reference therefore uses the direct resize (`stretch`), which is also the simplest one to reproduce on hardware. The remaining gap to the published figure is

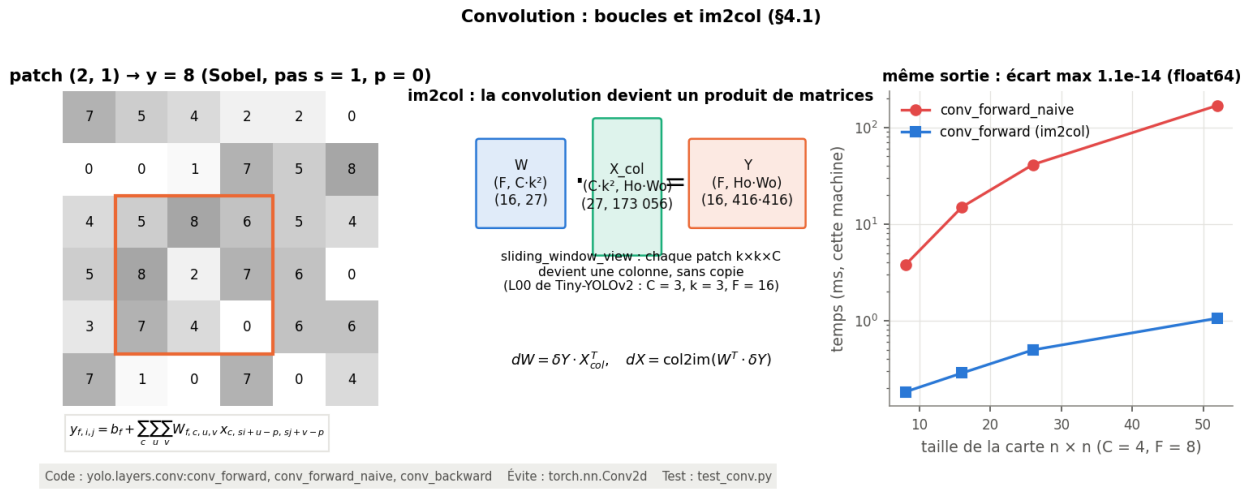


Figure 5: Convolution : la formule sur un patch, le dépliage im2col en produit de matrices, et le temps des boucles face à im2col. (`convolution`)

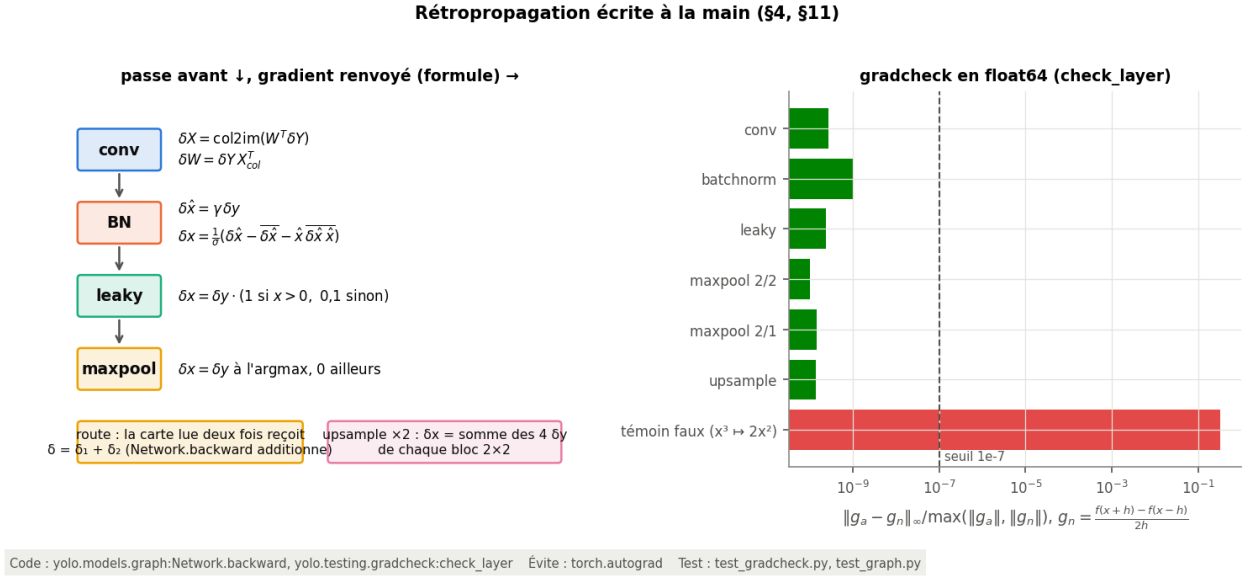


Figure 6: Rétropropagation écrite à la main : formule du gradient renvoyé par chaque couche, puis l'erreur du gradcheck par couche face au seuil 1e-7. (`retropropagation`)

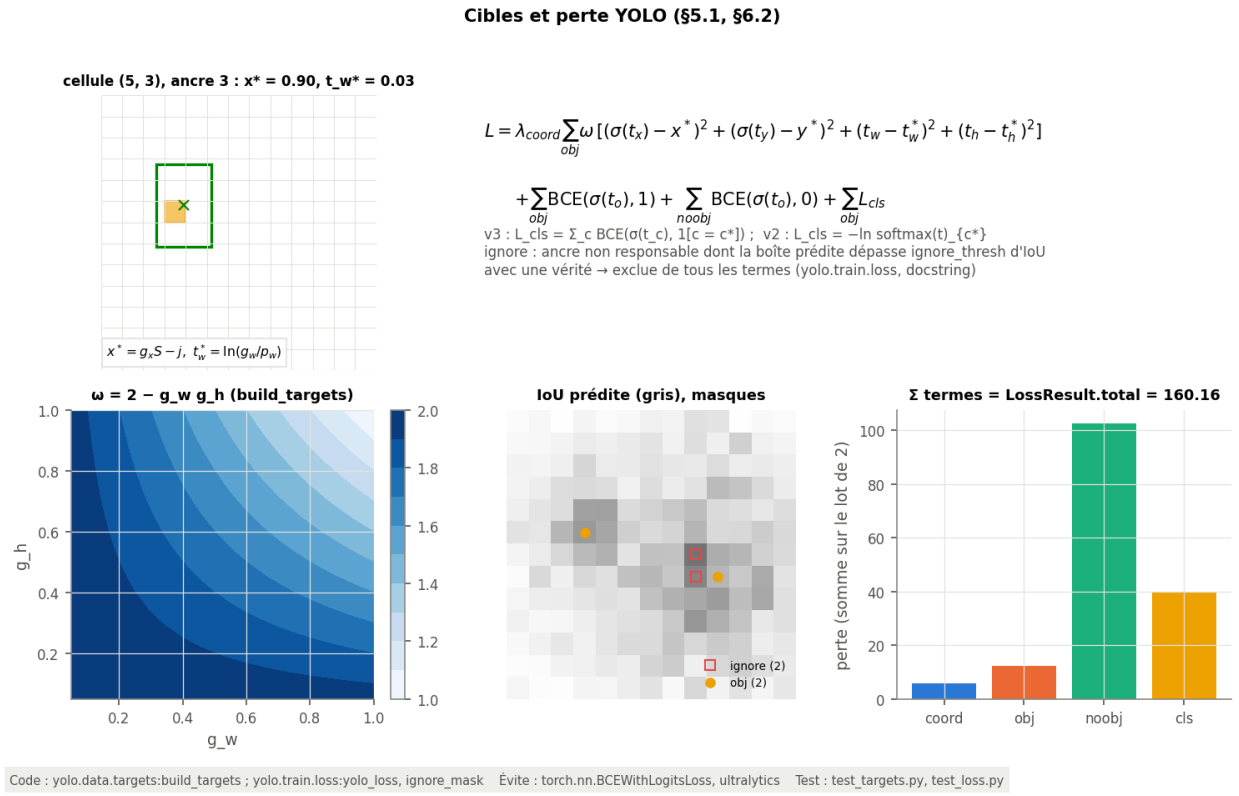


Figure 7: Cibles et perte YOLO : encodage d'une vérité, la perte terme par terme, le poids $\omega = 2 - \mathrm{g_w}\ \mathrm{g_h}$, le masque ignore et la part de chaque terme sur un lot. (`perte`)

below the one-point tolerance of the project; its plausible causes (JPEG decoder, details of the Darknet evaluation loop) are not measured. Figures 8 and 9 show the precision-recall curves and the evaluation steps.

Table 2: Float mAP on VOC2007 test by preprocessing (`results/map_float.md`).

Preprocessing	Interpolation	mAP
direct resize (`stretch`)	Pillow bilinear	56.30
direct resize (`stretch`)	Darknet (`resize_image`)	56.32
letterbox	Pillow bilinear	54.18
letterbox	Darknet (`resize_image`)	53.98

The COCO networks are evaluated on val2017 with the COCO metric (101-point interpolation, equal to `pycocotools` to 10^{-4}), same threshold and NMS (Table 3, Figure 10). The configuration of YOLOv4-tiny has no letterbox, and the direct resize is again the better preprocessing for it. Its float AP50, 41.4, is above the $40.2^{\text{pub.}}$ published on test-dev, a different image set. Tiny-YOLOv3 reaches 37.2 AP50, but its weights were trained on COCO 2014 images from which val2017 is partly drawn, so this value is optimistic and the comparison favours it.

5 INT8 quantization

The integer model follows the usual post-training quantization path [14, #008.0]. Batch normalization is first folded into the convolution weights and biases [4, #039.0]. Weights are quantized symmetrically per output channel; activations per layer, with scales calibrated on 500 VOC2007 trainval images (percentile `p99.99` of $|x|$, chosen by a mean-squared-error criterion). Each convolution accumulates in 32 bits and requantizes its output with an integer multiplier M_0 and a shift, so

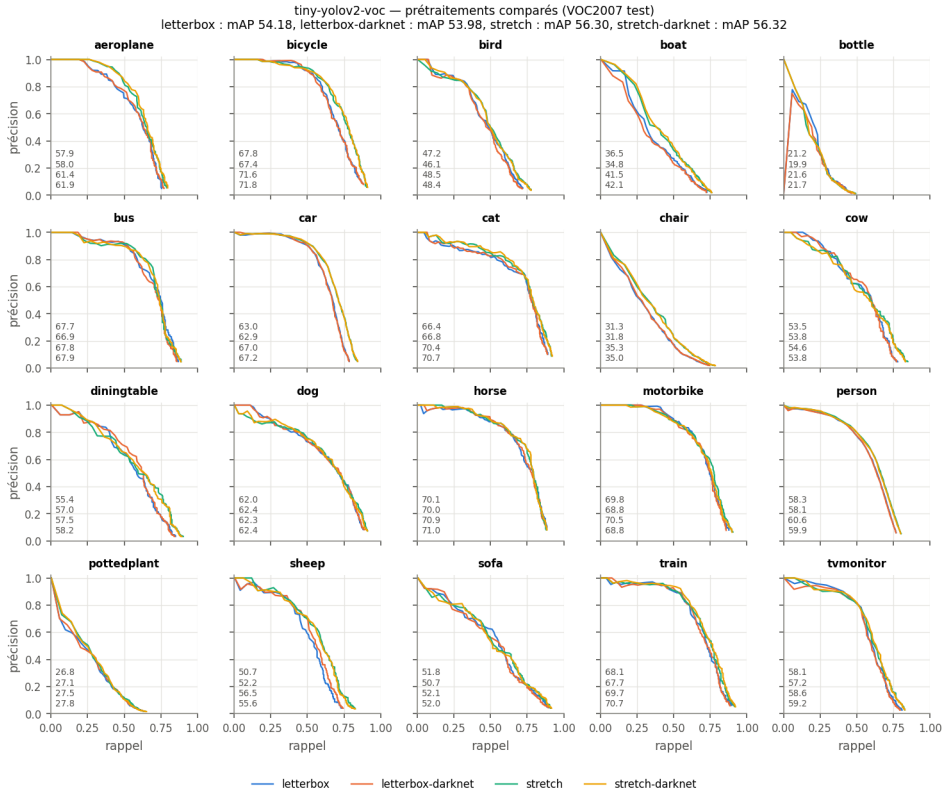


Figure 8: Courbes précision-rappel par classe (11 points VOC07) pour chaque prétraitement évalué, puis les quatre superposées. (`pr`)

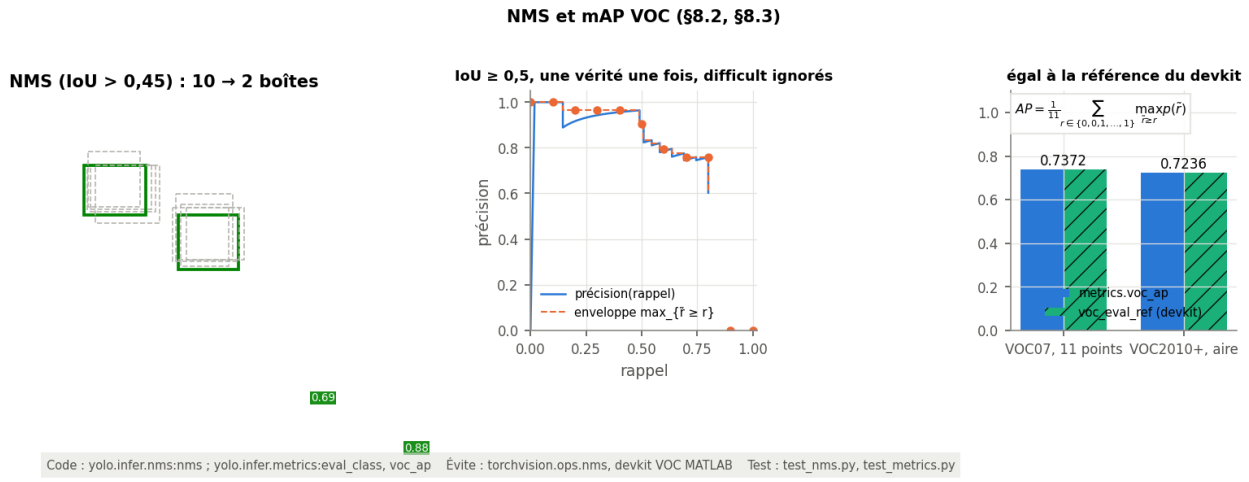


Figure 9: NMS pas à pas, appariement détections / vérités et courbe précision-rappel, AP VOC07 sur 11 points face à l'aire sous l'enveloppe, égale à la référence du devkit. (`nms_map`)

Table 3: Float AP on COCO val2017 (`results/map_coco.md`).

Network	Preprocessing	AP	AP50	AP75	APs	APm	APl
YOLOv4-tiny	stretch	21.7	41.4	20.5	9.9	26.0	30.7
YOLOv4-tiny	letterbox	21.2	39.9	20.4	8.1	24.2	33.6
Tiny-YOLOv3	stretch	16.7	37.2	12.7	5.7	19.0	26.1
Tiny-YOLOv3	letterbox	17.0	36.9	13.8	5.2	18.4	28.4

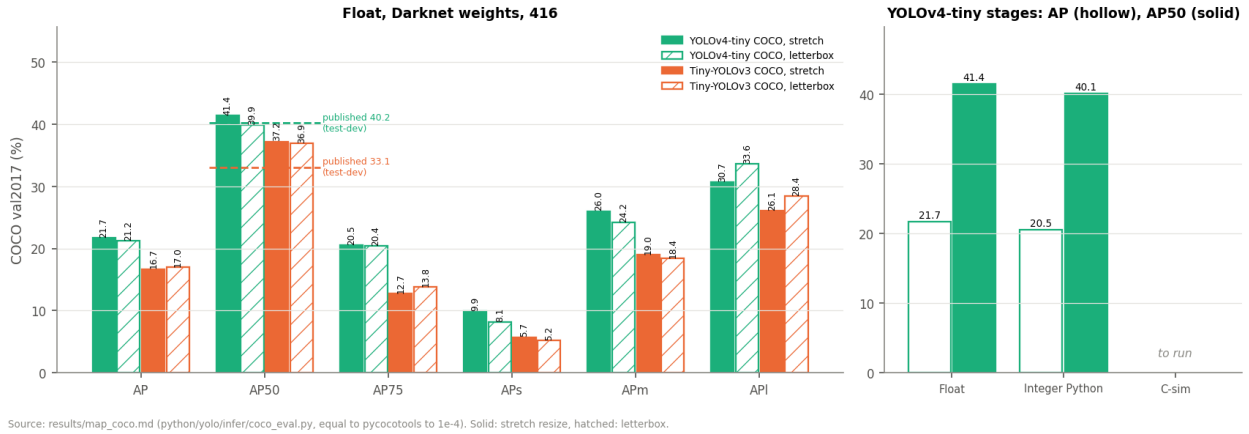


Figure 10: COCO val2017 AP of YOLOv4-tiny and Tiny-YOLOv3 (stretch and letterbox) with the published AP50, and the float / integer / C-sim stages of YOLOv4-tiny. (`map_coco`)

that no floating point remains in the network (outside the base). The leaky ReLU slope becomes 13/128, computed as $(13y + 64) \gg 7$: rounding instead of flooring avoids a bias of half a step on every negative value, which accumulates from layer to layer. The heads are decoded by 256-entry tables for the sigmoid and the exponential, as in 2024-zhang [8, #014.0]; their input step is 1/8, because at 1/16 the class logits of Tiny-YOLOv2 saturate and the softmax flattens (Figures 11, 12 and 13).

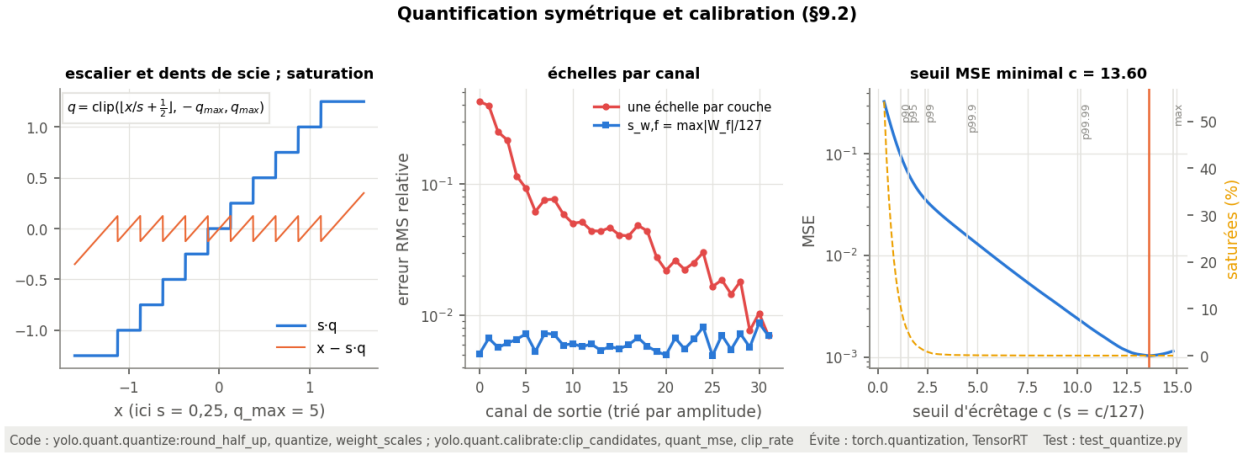


Figure 11: Quantification symétrique : l'escalier et son erreur, échelles par canal face à une échelle par couche, choix du seuil d'écrêtage par la MSE. (`quantification`)

The integer model stays within −0.64 point of the float model with the reference preprocessing (Table 4), below the one-point target: quantization-aware training is not needed. Quantizing one layer at a time on a subset of the test set, the most sensitive layer is $\mathrm{L10}^{\text{tier R}}$ ($-0.61^{\text{tier R}}$ point alone), of the same order as the noise of the subset: no single layer dominates the loss, which spreads over the network (Figure 14).

Table 4: mAP of the float and INT8 models on VOC2007 test (`results/map_int8.md`).

Preprocessing	Float (fused BN)	Integer INT8	Gap
stretch (`make eval-float`)	56.30	55.66	−0.64
letterbox (the training one)	54.18	53.68	−0.50

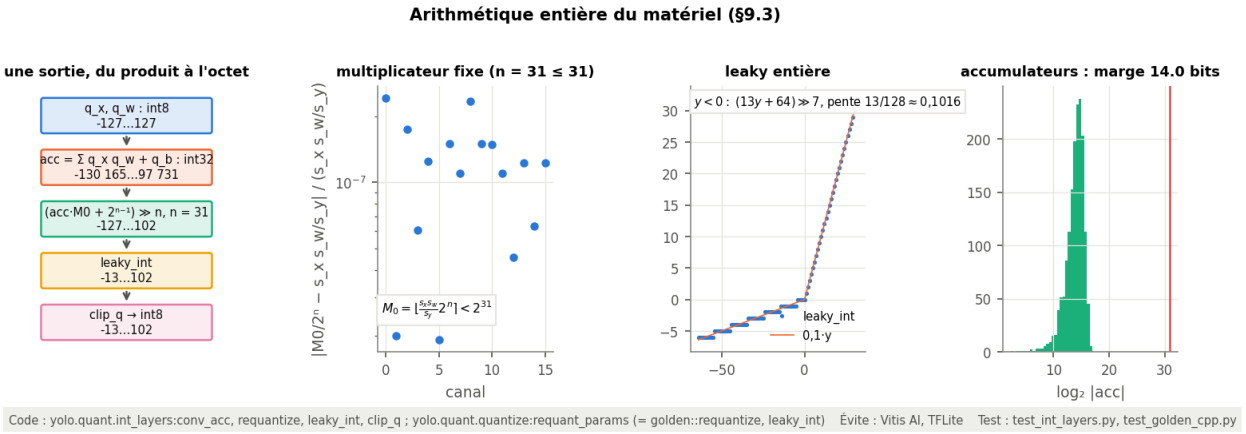


Figure 12: Arithmétique entière du matériel : chaîne d'une sortie, multiplicateur fixe $M0/2^n$, leaky entière 13/128 et marge des accumulateurs int32. (`entier`)

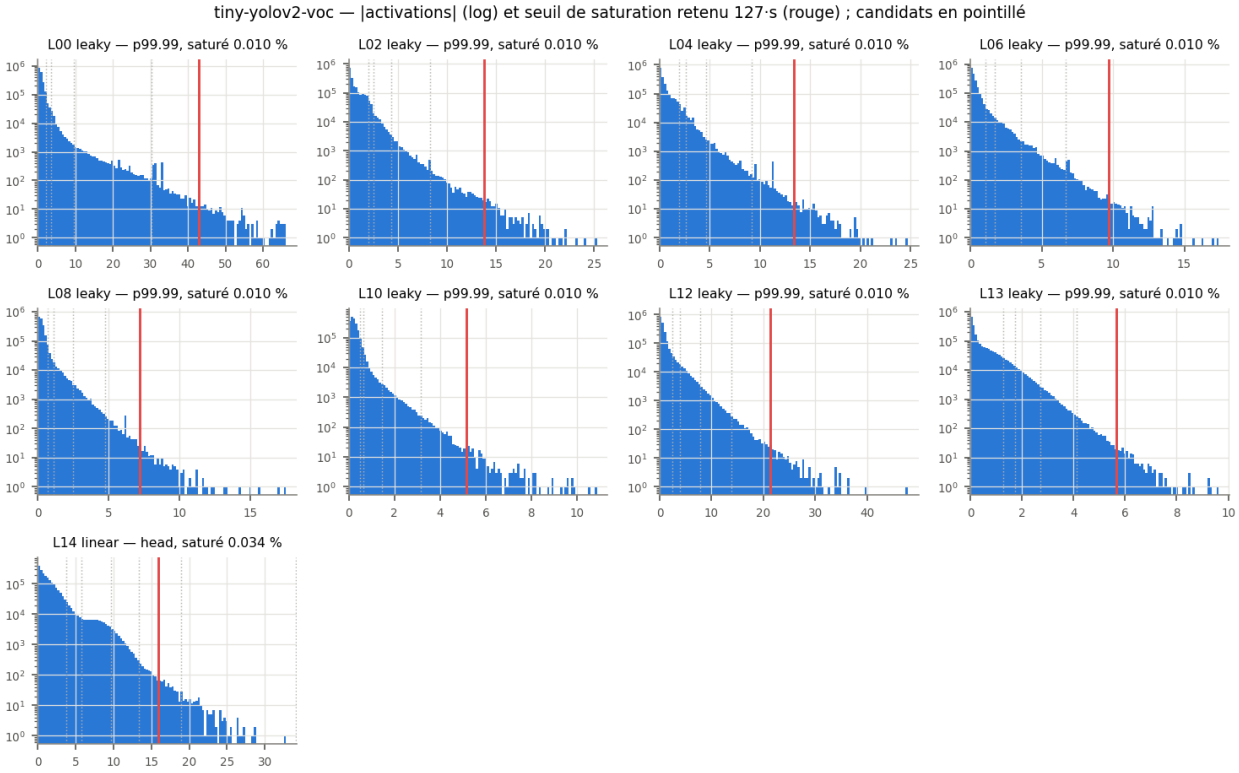


Figure 13: Distribution des activations par couche et seuil de saturation retenu par la calibration ; échelles des poids par canal. (`calibration`)

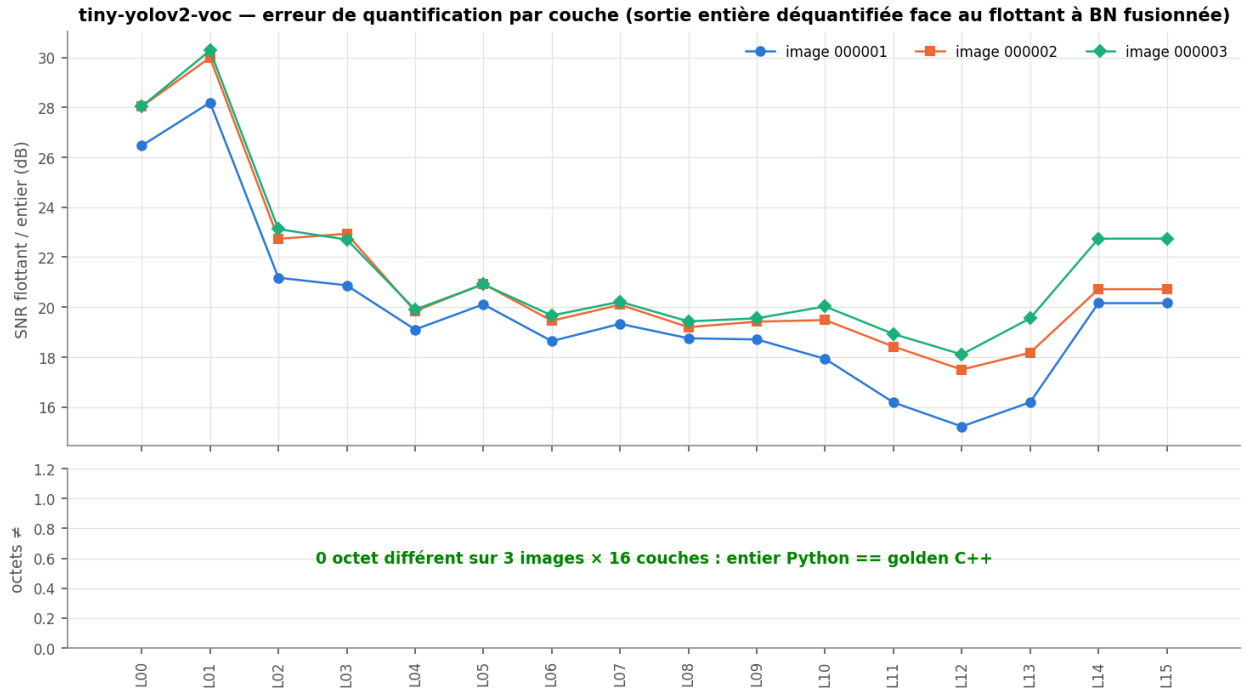


Figure 14: SNR par couche entre le flottant et l'entier déquantifié, et écart entier Python ↔ golden C++ (0 partout : bit-exact). (`erreur_couches`)

YOLOv4-tiny, calibrated in the same way on 500 COCO train2017 images, loses more: −1.3 point of AP50 (40.1 in integer), above the one-point target. Fake-quantizing the two heads alone costs nothing, so the loss is again spread over the backbone; a coarser head scale would clip less but overflows the exponential table of the post-processing. Quantization-aware training is left as follow-up.

6 Golden C++ model and verification chain

The quantized network is exported as a manifest and binary blobs (`model/<net>/`), with the integer activations of every layer on a few images as dumps. A C++ golden model reads the same export and runs the network *as the hardware will*: tiled convolution with fixed-size buffers, output stage with bias, requantization, leaky and fused max pooling, route by addressing. It is the testbench of the HLS stage.

Equality is checked at every hand-over (Figure 15). `make golden-check` runs the golden model on 12 dumps (three images per quantized network) and compares every layer to the Python dumps; the HLS kernel in C simulation is compared to the golden model layer by layer (`make csim-gcc`); the ARM driver, built with a simulation backend that calls the C-simulated kernel, is compared to the golden model on the final heads (`make sw-sim`). The integer post-processing kernel of Section 9 is compared to its golden mirror on $1{,}014^{\text{C-sim}}$ cases. All of these run in `make ci` on every commit, on a synthetic export when the Darknet weights are absent.

7 HLS accelerator and SoC integration

The board was chosen with a roofline explorer before writing any HLS (ADR 0003): the Kria KV260 offers, at the same price, about six times the roofline performance of a Zynq-7020 board for these networks. The accelerator is a single layer-by-layer engine (Figure 16): the convolution loop nest is tiled over output channels, input channels, rows and columns [7, #010.2], with $T_m = 32$, $T_n = 24$

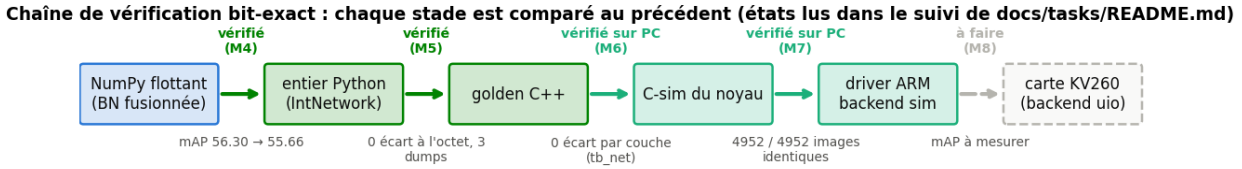


Figure 15: Chaîne de vérification bit-exact, du flottant NumPy à la carte : critère et état de chaque passage. (`chaine_verif`)

and $T_r = T_c = 13$, i.e. 768 MACs per cycle at 200 MHz (Figure 17). One engine runs every layer of both networks; upsample and route need no compute, they change the read address of the next layer.

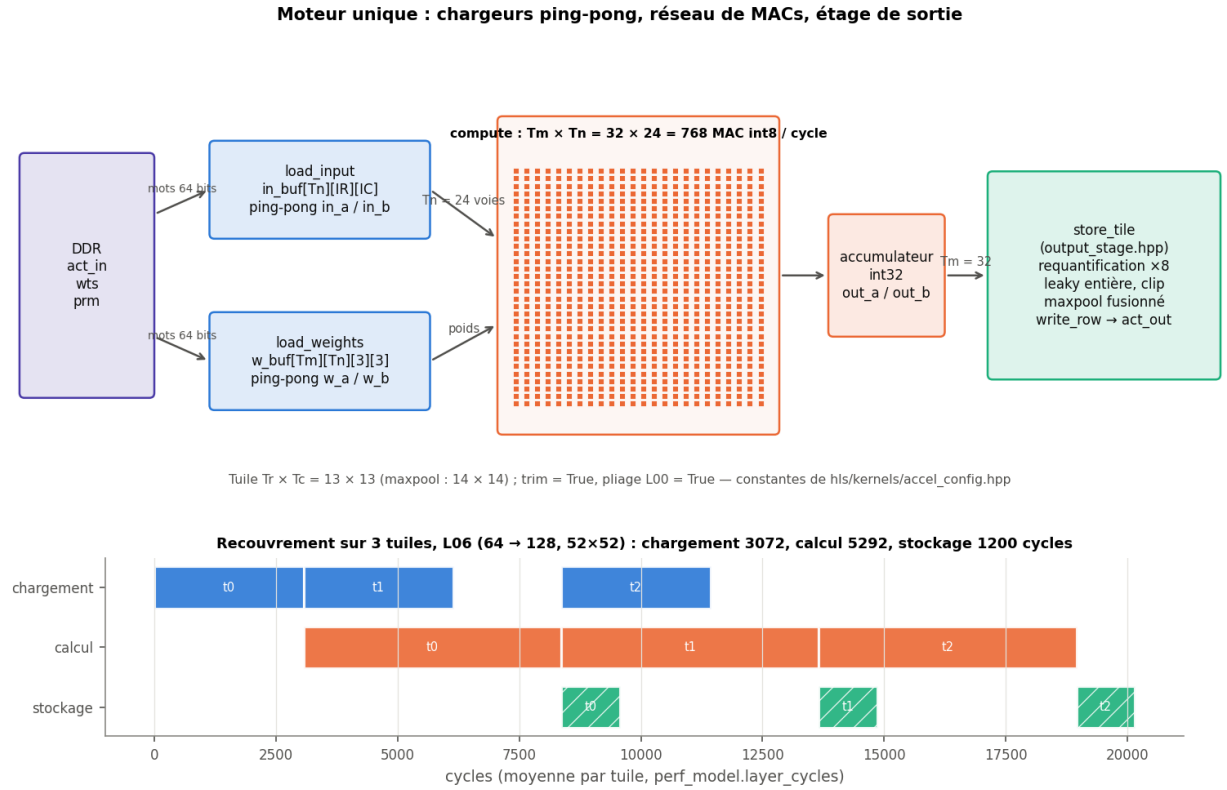


Figure 16: Moteur unique : chargeurs ping-pong, réseau Tm × Tn de MACs, accumulateur, étage de sortie, et chronogramme du recouvrement sur 3 tuiles. (`moteur`)

The ARM side loads the manifest and the blobs in contiguous memory, programs the AXI-Lite registers of the kernel layer after layer and waits for its interrupt (Figure 18). The block design, the device-tree overlay and the firmware packaging are scripted (`make vivado-build`, `make fpga-firmware`) but have not run yet: Vivado and Vitis are not installed on the development machine. Everything above the hardware runs on the PC through the simulation backend.

A cycle model in Python (`tools/perf_model.py`) reproduces the cycle counters of the C-simulated kernel exactly, layer by layer (`make perf-model` checks it in CI). It shows where the first kernel (M6) lost its time: with 8-bit memory ports it spent most of its cycles outside the MAC loop, waiting for its ports and its output stage, and its projection was $206.5^{\text{proj.}}$ ms per image. The optimization cascade of milestone M10 (Table 5, Figure 19) widens the ports to 64 bits, loads only the valid input channels, requantizes eight channels per cycle, folds the first layer onto the input lanes and uses 14 × 14 tiles for the pooled convolutions; each step is C-sim equal to the golden model. The current kernel projects $37.2^{\text{proj.}}$ ms, against a compute bound of $31.6^{\text{proj.}}$ ms and a roofline point of $31.9^{\text{proj.}}$ ms (Figures 20 and 21).

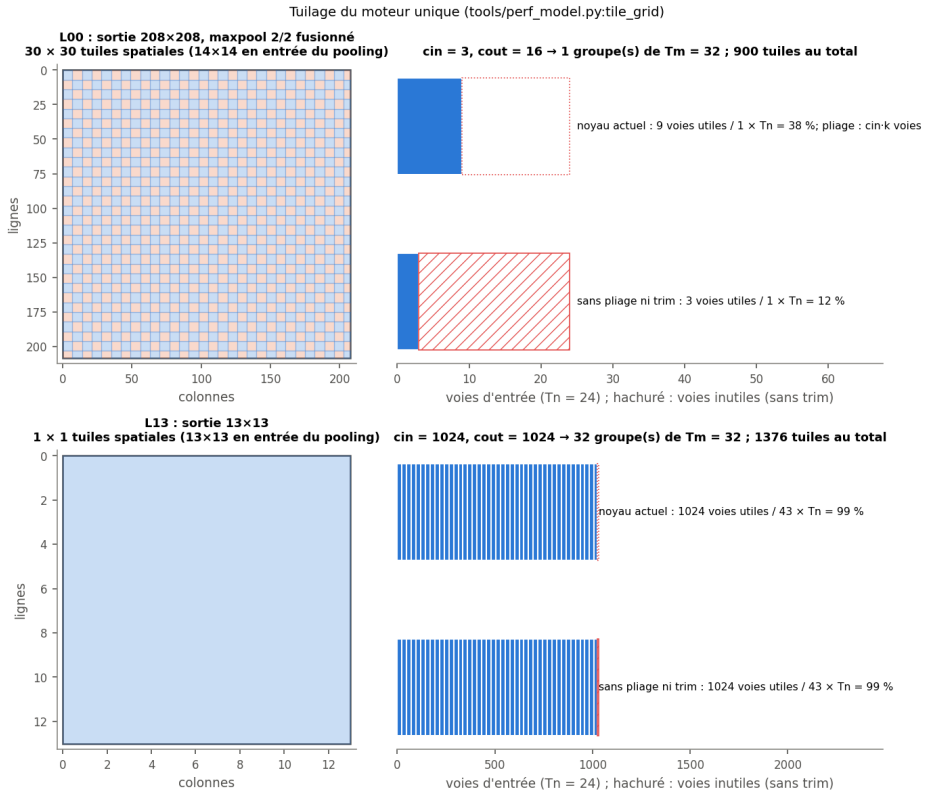


Figure 17: Tuilage de deux couches : tuiles spatiales et voies d'entrée, avec le pliage de L00 (cin = 3) et la couche 13×13 couverte par une seule tuile. (`tuilage`)

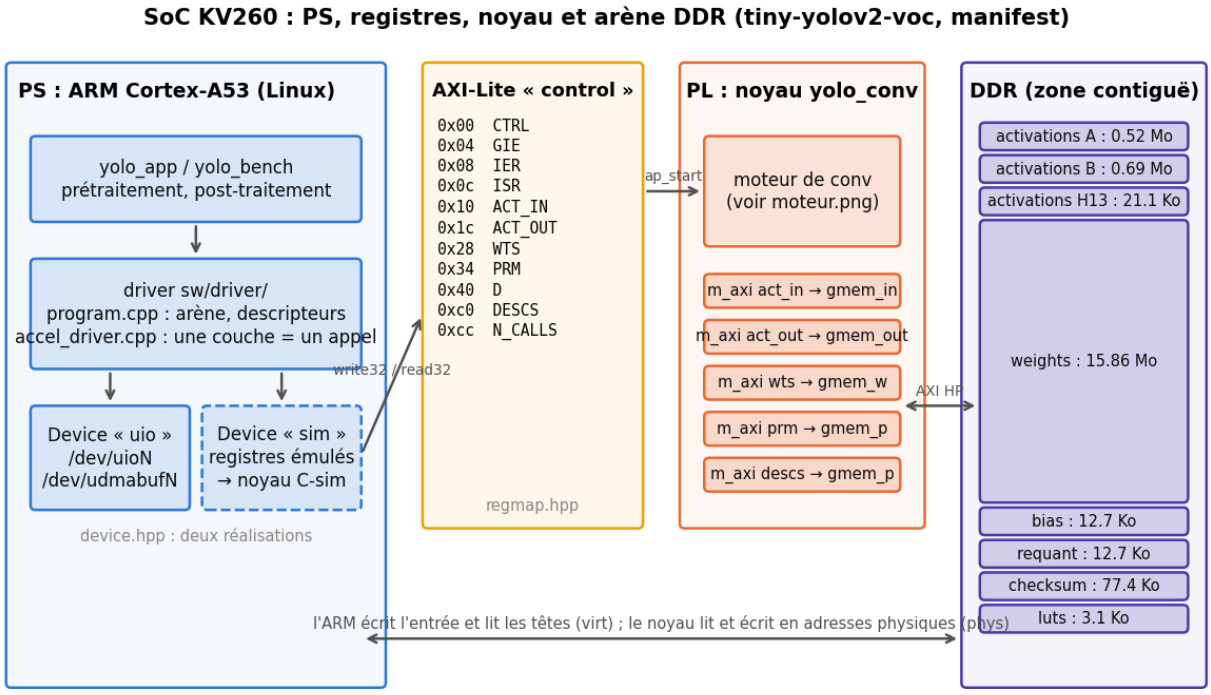


Figure 18: Schéma du SoC : application et driver sur l'ARM, registres AXI-Lite, ports m_axi du noyau et découpage de l'arène DDR. (`soc`)

Table 5: Optimization cascade of the kernel, Tiny-YOLOv2, accelerator only (`results/rapport.md`).

Kernel configuration (Tiny-YOLOv2)	Mcycles	ms	img/s	GOPS	MAC efficiency (%)
M6: ports 8 bits	41.30	206.5	4.8	33.8	11.0
valid channels only (trim)	32.67	163.3	6.1	42.7	13.9
ports 64 bits	12.27	61.3	16.3	113.6	37.0
ports 128 bits + trim	12.13	60.6	16.5	115.0	37.4
ports 64 bits + trim + requantization ×8 (+28 DSP)	9.29	46.4	21.5	150.1	48.9
... + L00 folding	8.38	41.9	23.9	166.4	54.2
... + 14×14 tiles for pooled convs (M10 kernel)	7.44	37.2	26.9	187.3	61.0
compute bound of M6 (free loads)	8.23	41.1	24.3	169.5	55.2
compute bound of the M10 kernel	6.32	31.6	31.7	220.7	71.8
KV260 roofline point (roofline.md)	—	31.9	31.3	218.5	—
ideal (MACs / 768, efficiency 100 %)	4.54	22.7	44.1	307.2	100.0

Status of every number of this table: projection.

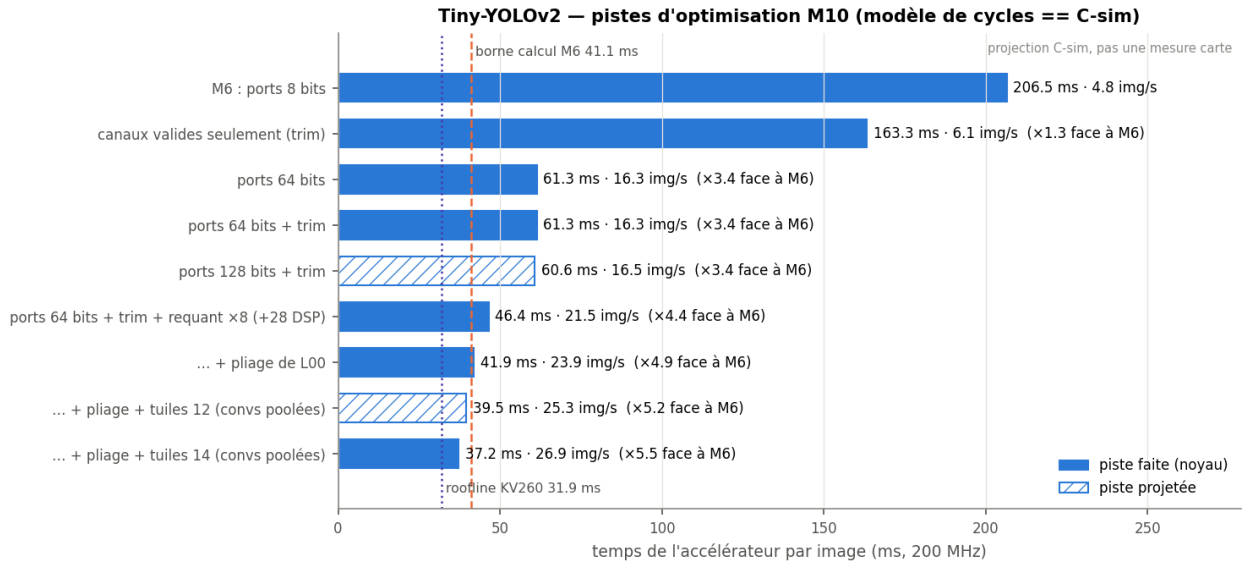


Figure 19: Temps par image de Tiny-YOLOv2 à chaque piste d'optimisation M10, face à la borne de calcul et au roofline KV260. (`cascade_m10`)

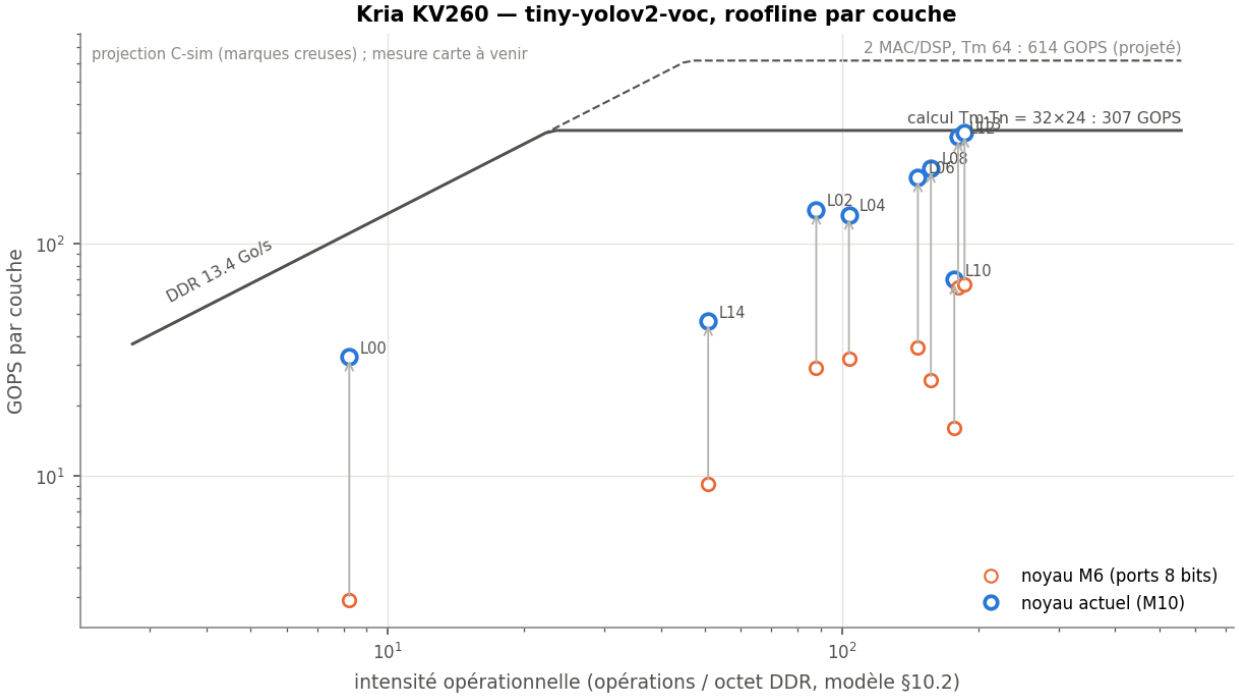


Figure 20: Roofline KV260 avec un point par couche de Tiny-YOLOv2, avant et après les optimisations M10. (`roofline_couches`)

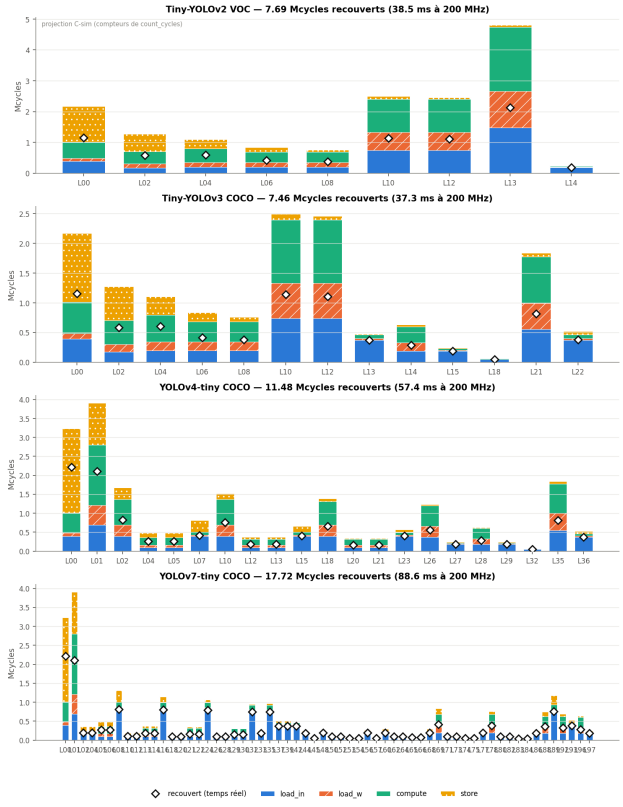


Figure 21: Cycles par couche du noyau actuel, décomposés en chargements, calcul et stockage ; le losange donne le temps réel avec recouvrement. (`cycles_couches`)

Two INT8 multiplications per DSP. The compute bound only falls if the multiplications per cycle rise. As in 2024-zhang [8, #011.0], the weights of two output channels that share an input are packed into the 27-bit port of a DSP48E2, $A = w_a 2^{18} + w_b$, and one product gives both (Figure 22). The low lane stays exact for a sum of eight products under the contract of the integer model (weights and inputs within ± 127), so the 24-lane adder tree is cut into three groups of eight, each unpacked to int32. Only the multiplication changes: the C-simulated kernel equals the golden model byte for byte on every testbench, and its cycles are unchanged. With the same tiles, packing frees half of the DSPs; with $T_m = 64$ on the same DSPs it projects $24.7^{\text{proj.}}$ ms for Tiny-YOLOv2, but doubles the weight registers beyond the flip-flops of the board unless they move to LUTRAM, and $T_n = 32$ gives $30.0^{\text{proj.}}$ ms within the budget (Table 6, Figure 23). The YOLOv4-tiny projection with the current kernel is $51.8^{\text{proj.}}$ ms (Table 9).

Table 6: Packed PE options, accelerator only, KV260 at 200 MHz (`results/dsp_packing.md`).

Network	Option	Tm, Tn	PE DSP	ms	GOPS
Tiny-YOLOv2 VOC	M10 kernel, 1 MAC / DSP	32, 24	768	37.2	187.3
Tiny-YOLOv2 VOC	packed, same tiles (half the DSPs)	32, 24	384	37.2	187.3
Tiny-YOLOv2 VOC	packed, Tm 64 (same DSPs)	64, 24	768	24.7	282.5
Tiny-YOLOv2 VOC	packed, Tn 32 (roofline point)	32, 32	512	30.0	232.2
Tiny-YOLOv3 COCO	M10 kernel, 1 MAC / DSP	32, 24	768	36.1	154.3
Tiny-YOLOv3 COCO	packed, same tiles (half the DSPs)	32, 24	384	36.1	154.3
Tiny-YOLOv3 COCO	packed, Tm 64 (same DSPs)	64, 24	768	23.8	233.4
Tiny-YOLOv3 COCO	packed, Tn 32 (roofline point)	32, 32	512	30.2	184.0

Status of every number of this table: projection.

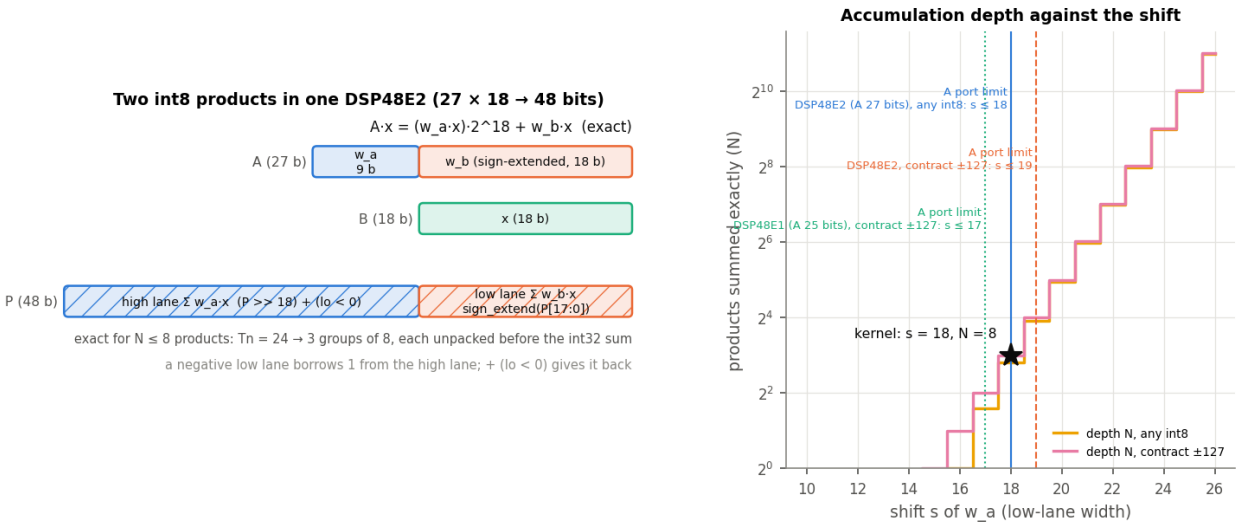


Figure 22: Two int8 MACs per DSP (M23): bit layout of the A port and of the product, and accumulation depth N against the shift s with the port limits. (`dsp_packing`)

Winograd. An integer-exact Winograd $F(2 \times 2, 3 \times 3)$ removes more than half of the multiplications of the 3×3 convolutions [4, #004.0] and is slightly ahead of packing on the compute bound alone. Once the loads are counted it is not (Table 7, Figure 24): the transformed weights are wider and consumed faster, its weight buffer exceeds the flip-flops of the board, and its products do not fit two to a DSP. It is studied, not implemented.

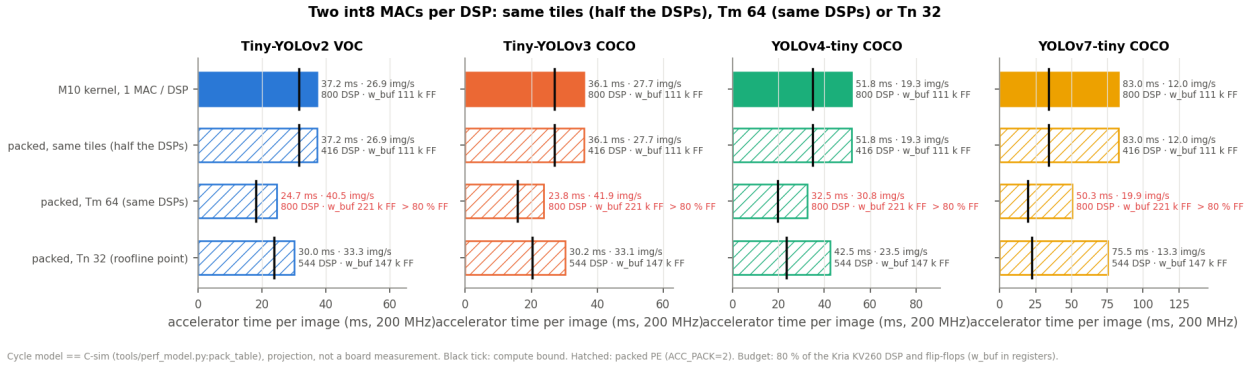


Figure 23: Time per image of each network with the packed PE (T23.3): M10 kernel, same tiles, Tm 64 and Tn 32, with the compute bound, DSP and w_buf flip-flops. (`pack_options`)

Table 7: Winograd against packing, milliseconds, Tiny-YOLOv2 and Tiny-YOLOv3 (`results/winograd.md`).

Network	Bound: direct	Winograd	Packing	With loads: Winograd offline	on the fly	Packing
`tiny-yolov2-voc`	31.6	17.1	18.1	42.8	22.8	19.8
`tiny-yolov3-coco`	27.0	14.8	15.8	38.8	23.0	18.8

Status of every number of this table: estimate.



Figure 24: Winograd F(2×2, 3×3) against the packed PE: compute bound, time with the loads and weight footprint of the 3×3 convs, per network. (`winograd`)

8 Results

Table 8 gives the mAP at the three stages of the chain. The float model and the Python integer model are evaluated on the full test set; the FPGA stage runs the ARM driver over the C-simulated kernel, compiled with native integers (same arithmetic as `ap_int`, which was also checked on part of the set). Its detections are identical to those of the integer model image by image, so the whole accuracy loss comes from quantization, and the board will give the integer mAP if `run_compare` confirms zero mismatch there (Figure 25).

Table 8: mAP at the stages of the chain, VOC2007 test (`results/map_stades.md`).

Stage	mAP	Equality with the integer model	Status
Float (fused BN, NumPy float32)	56.30	—	measured
Integer, Python (`IntNetwork`, bit-exact)	55.66	reference	measured
FPGA, C-sim (ARM driver, sim backend, PC)	55.66	4,952 / 4,952 identical images	csim
FPGA, KV260 (uio backend)	—	to be measured	—

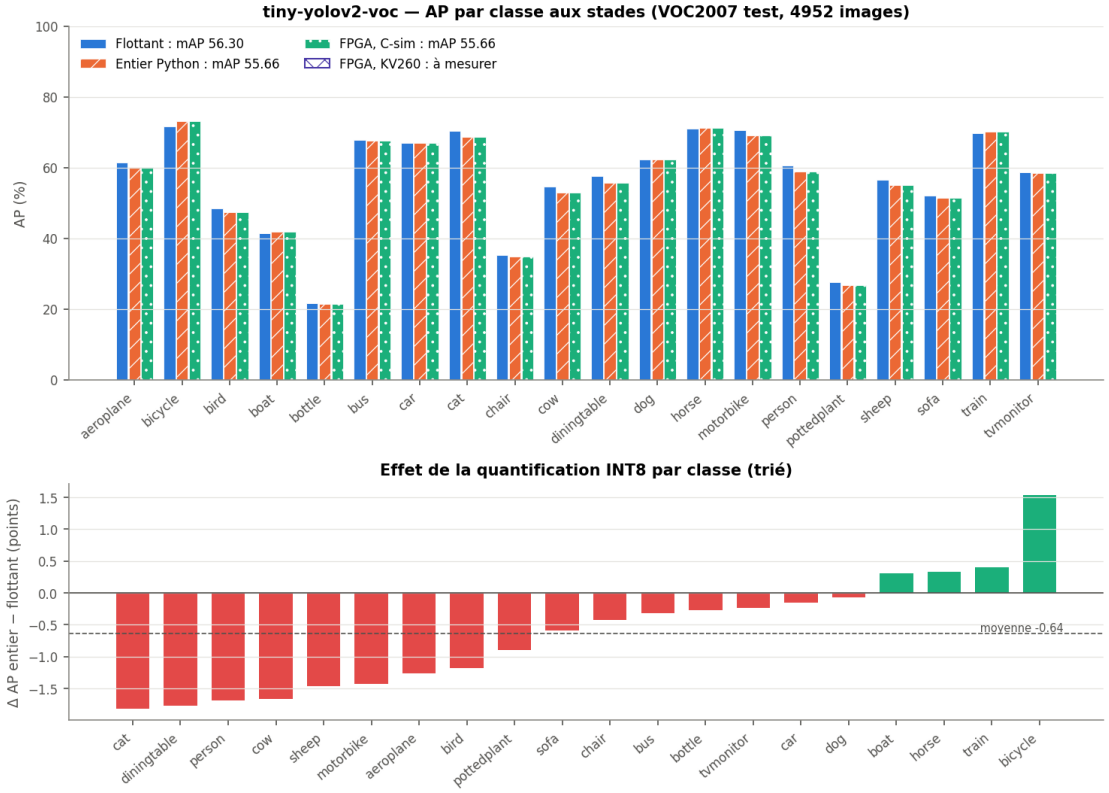


Figure 25: AP par classe aux stades flottant, entier et C-sim (la carte quand elle sera mesurée), et écart entier − flottant trié. (`map_stades`)

Table 9 gives the performance of the accelerator. These are projections of the cycle model, a lower bound of the accelerator time: they assume an initiation interval of one in the inner loop and ignore pipeline depths, DDR latency and the ARM control. Resources will only be known after synthesis; Figure 26 compares the roofline model with the streaming plan of Section 9.

Against 2024-zhang, the only reference on the same model, packing closes the gap in multipliers (Section 7); what remains is the share of time our kernel still spends outside the MAC loop, in the

Table 9: Projected performance on the KV260 at 200 MHz (`results/mesures.md`).

Network	GMAC	Mcycles	ms	img/s	GOPS	MAC efficiency (%)	Status
`tiny-yolov2-voc`	3.486	7.44	37.2	26.9	187.3	61.0	projection
`tiny-yolov3-coco`	2.782	7.21	36.1	27.7	154.3	50.2	projection
`tiny-yolov4-coco`	3.454	10.36	51.8	19.3	133.3	52.3	projection

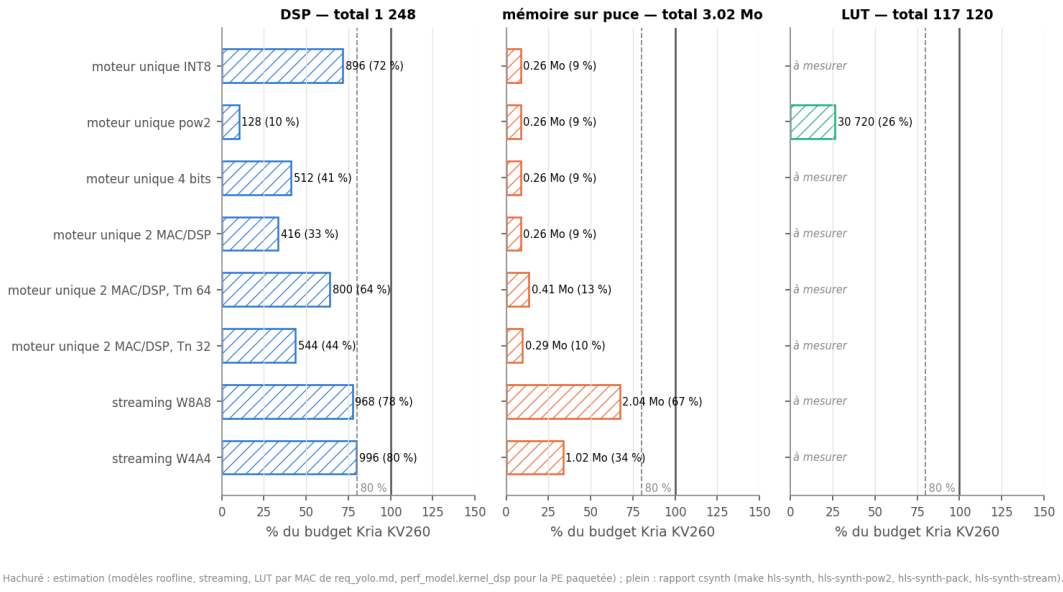


Figure 26: DSP, mémoire sur puce et LUT des architectures (moteur unique INT8, pow2, 4 bits et 2 MAC/DSP, streaming) face au budget de la KV260 ; hachuré : estimation, plein : synthèse. (`ressources`)

weight and input loads of the 1×1 and 13×13 layers and in the output stage of the first layer. On the same board, the projected throughput of the current kernel exceeds the DPU of 2026-fata in images per second, without pruning (Table 1).

9 Extensions

Hardware post-processing. Following 2024-zhang [8, #013.1, #014.0], a separate HLS kernel decodes the heads and runs the NMS in integers only: the objectness threshold is applied to the integer logit, the softmax uses one reciprocal per cell, box corners are Q4 pixels, the IoU test becomes an integer inequality, and the NMS keeps 256 slots in stream order without sorting. This NMS does not always give the result of the sorted one (a chain of three overlapping boxes keeps only one); on the test set the integer post-processing reaches 55.56 mAP (Figure 27). The kernel equals its golden mirror in C simulation; its synthesis is pending.

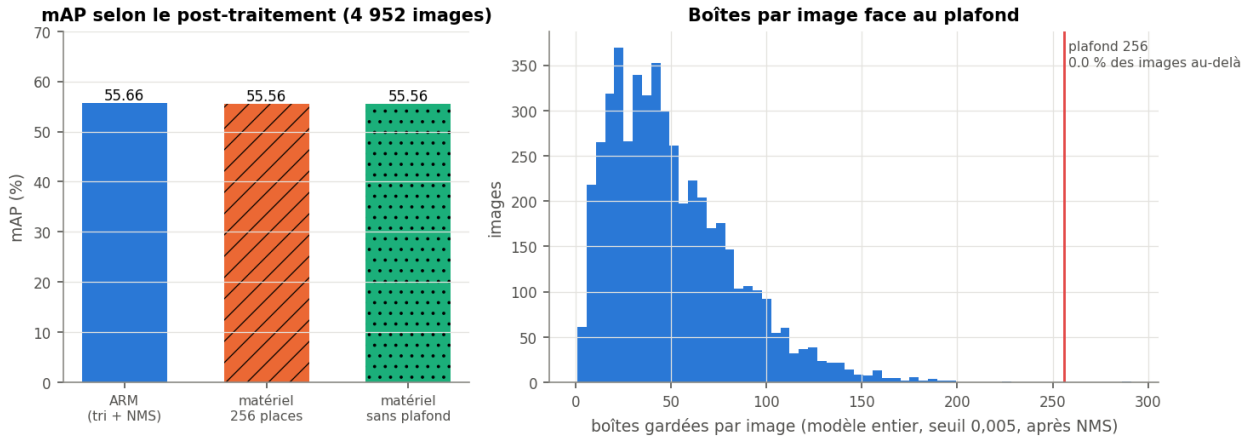


Figure 27: mAP avec le post-traitement matériel, avec et sans plafond de boîtes, et nombre de boîtes par image face au plafond. (`hw_postproc`)

Power-of-two weights (REQ-YOLO). We reproduce the "mixed powers of two" levels of 2019-ding [9, #008.1]: each weight is a sum of at most two powers of two, so a multiplication becomes two shifts and an addition, and the engine needs no DSP for its MACs (estimate). The direct projection of the trained weights gives 52.46 mAP, against 54.35 for uniform 6-bit levels. The ADMM fine-tuning [9, #009.1] *did not converge*: the residual between the weights and their projection grew instead of vanishing, the detection loss rose, and the result, 43.07 mAP, is worse than the projection alone. The likely causes (fine-tuning without BN, small batches, a penalty too weak) are not verified. The power-of-two variant of the PE is C-sim equal to the golden model.

4-bit weights and activations. With 4-bit weights per channel and 4-bit activations on power-of-two steps [15, #006.0], keeping the input and the head on 8 bits, post-training quantization collapses to 17.06 mAP. A short quantization-aware training (600 iterations, a fraction of an epoch, on CPU) recovers part of it, to 36.38 mAP; the loss was still decreasing, so a longer run should do better. The 4-bit export is reproduced byte for byte by the golden model and the C-simulated kernel. Table 10 and Figure 28 sum up the formats.

Streaming architecture. The second family of Section 2 [5, #004.0] is planned with one stage per convolution and line buffers [6, #011.0]. The weights of Tiny-YOLOv2 do not fit on the chip of the KV260, so the plan is hybrid: the small layers stream with their weights on chip, the two largest read theirs from DDR once per image. The resource model estimates $32.1^{\text{est.}}$ images/s with

Table 10: Formats on the same model, VOC2007 test (`results/req_yolo.md`, `quant_4bits.md`, `postproc_hw.md`).

Weights / activations	Method	mAP	Gap to INT8
INT8 per channel (reference)	PTQ	55.66	—
uniform6 (±31)	PTQ	54.35	−1.31
mixed6 (powers of 2)	PTQ, direct projection	52.46	−3.20
mixed6	ADMM, 600 iterations, then projection	43.07	−12.59
w4a4	PTQ (power-of-2 steps)	17.06	−38.60
w4a4	QAT, 600 iterations	36.38	−19.28
INT8 + hardware post-processing	unsorted NMS, Q4 boxes	55.56	−0.10

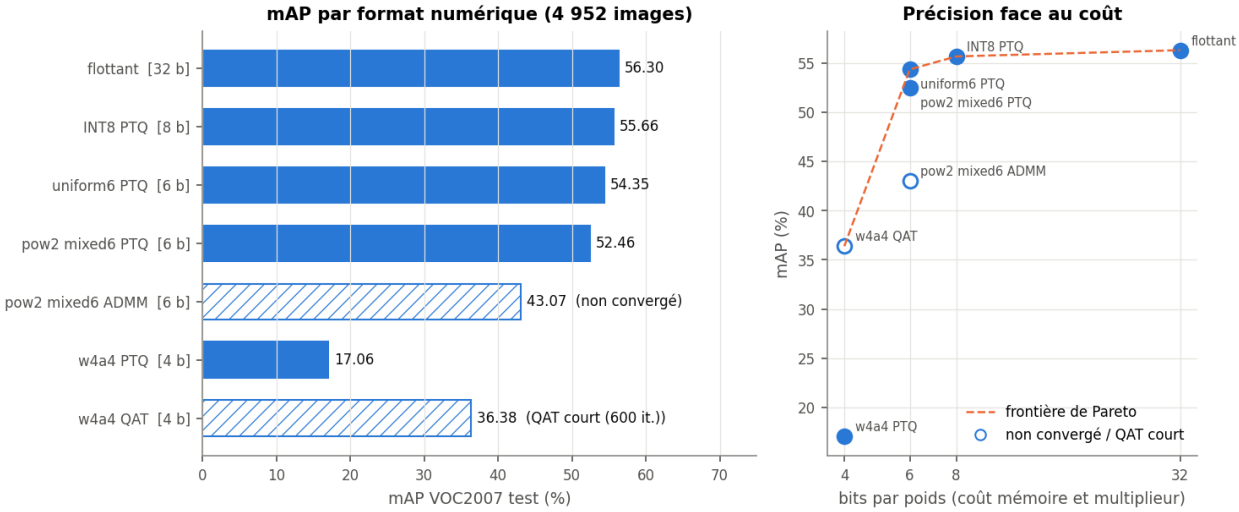


Figure 28: mAP de Tiny-YOLOv2 selon le format des poids (flottant, INT8, puissances de 2, 4 bits), et frontière précision / bits. (`map_formats`)

968$^{\text{est.}}$ DSPs, at a latency of 85.2$^{\text{est.}}$ ms, and 64.2$^{\text{est.}}$ images/s with 4-bit weights (Figure 29). The streaming kernel equals the golden model in C simulation, stage by stage; these figures are estimates until synthesis.

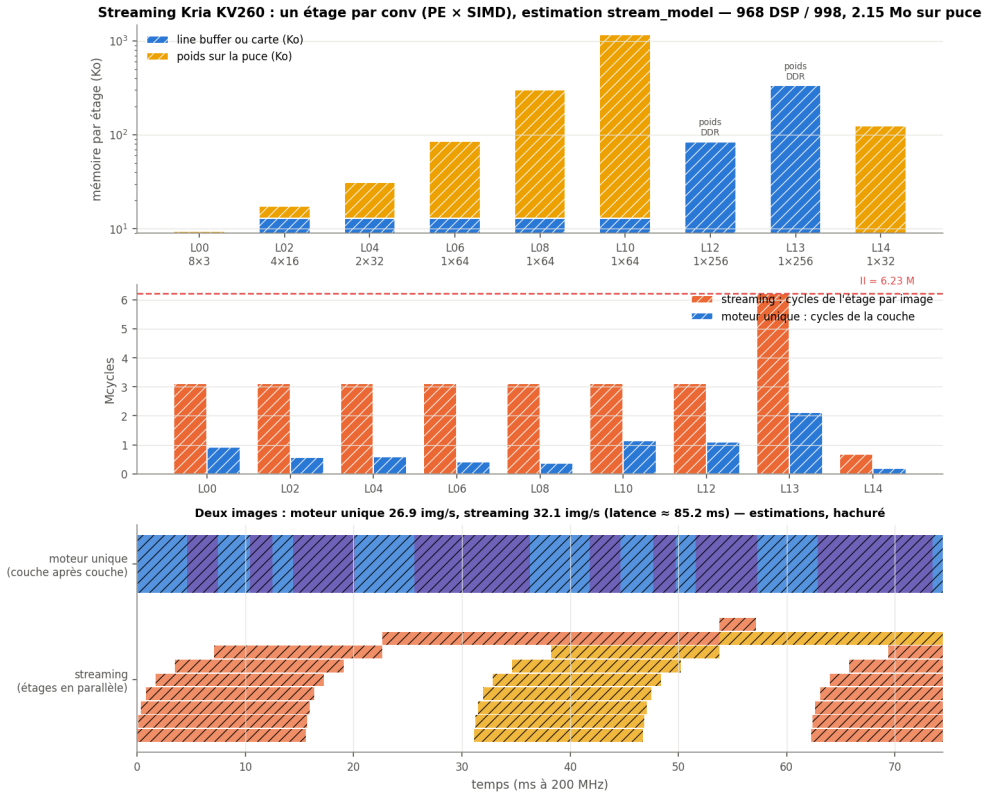


Figure 29: Architecture streaming face au moteur unique : mémoire et parallélisme par étage, cycles par couche, chronogramme sur deux images (estimations). (`streaming`)

10 Dependability

The bit-exact integer model is a fault-free reference: any difference in its output is caused by an injected fault. This section is engineering outside the base.

Fault injection. An injector flips bits in the INT8 weights, the int32 biases, the INT8 activations and the int32 accumulator of a chosen layer, either one given bit or at random with a bit error rate, or holds them stuck at a value; every run is seeded, and with no fault the output equals the integer model exactly. A campaign tool sweeps the error rate, the layer and the bit position against the mAP. Only a smoke run has been made so far; the full grids are to run.

Exact checksum (ABFT). The sum of the output channels of a convolution equals the convolution of its input by the sum of its filters. Before requantization everything is an integer, so the check holds exactly modulo 2^{32}: no tolerance and no false alarm. The summed filter and bias are exported with each layer, and the check runs in the integer model, in the golden model and in the HLS kernel. The kernel computes the checksum as one more output lane in the same pipelined loop and compares it in the requantization pass; the failing pixels go to a status register read by the ARM driver. Without fault, every bit-exact test passes with no ABFT error (103$^{\text{C-sim}}$ layers of the four exports, 158$^{\text{C-sim}}$ layer outputs of the network test bench, in C simulation), and every injected accumulator fault is detected. The cost of the lane waits for synthesis.

Faults in the kernel buffers. A fault hook of the C simulation, absent from the synthesis, flips one bit per kernel run in an on-chip buffer of the convolution: input tile, weights, int32 accumulators, requantized tile or checksum lane. A Python reference maps each buffer address back to its tensor element through the lane mapping of the kernel and recomputes the outputs of the tile: the faulty outputs match it byte for byte, and the outputs outside the tile equal the fault-free dump.

Out-of-distribution monitor. The monitor reads the INT8 output of the last backbone layer of Tiny-YOLOv2, which the accelerator already writes, and scores each image on the ARM. Dark (ExDark) and thermal (FLIR) images give *weaker* features than VOC: they fall inside the shell on which in-domain features lie, so a raw distance score is lower out of domain. The monitor therefore uses the typicality of the score, its distance to the median on held-out VOC images in either direction. The plain norm of the pooled features is the best score (AUROC 0.74 on low light, 0.88 on thermal), ahead of a Mahalanobis distance and of a NumPy autoencoder that costs ten times more (Table 11). It is a domain alarm, not a per-image predictor of accuracy.

Table 11: OOD monitor on the features of the integer Tiny-YOLOv2, typicality scores, VOC test against ExDark and FLIR (`results/ood_monitor.md`).

Score	AUROC ExDark	AUROC FLIR	Spearman score / mAP	MACs (share of the detector)
norm	0.744	0.884	−0.66	0.35 M (0.01 %)
Mahalanobis	0.669	0.726	−0.41	4.54 M (0.13 %)
autoencoder	0.686	0.718	−0.14	45.69 M (1.31 %)

11 Safety evidence

Dependability detects faults at run time; this section collects the design-time evidence that the circuit computes the model, and how long it may take. It is engineering outside the base.

Model ↔ circuit equivalence. Each hand-over of the chain is qualified at one of three levels: *pointwise* (same output), *bounded proximity* (a measured bound on the gap) or *statistical* (same task metric). The float model and its fake-quantized copy differ by a few LSB of the layer scale, which accumulate through the layers; the statistical gap of Section 5 is the one that counts. From the fake-quantized model to the integer model, the only difference is the requantization by an integer multiplier and shift and the rounded leaky activation: at most one LSB per layer. From the integer model on, the golden model and the C-simulated kernel produce the same byte for every layer output (Table 12). This pointwise level rests on three further pieces of evidence: exhaustive and boundary tests of the operators in Python, C++ and HLS, which found and fixed an int32 overflow of the golden leaky; a proof with the z3 SMT solver of the requantization, leaky and clipping kernels for every layer (95 obligations, no int32 overflow of the accumulator); and a per-layer traceability table checked in `make ci`. The integer post-processing equals its golden mirror and keeps its capacity and overflow properties, but its unsorted NMS depends on the order of the cells. The co-simulation and board rows wait for Vitis and the KV260.

A latency bound independent of the image. The engine runs a fixed schedule of layer descriptors: every loop bound comes from a descriptor field or a tile constant, and activations and weights are never a loop or branch condition. The cycle count of a layer is therefore a function of its shape only. The static bound gives up every overlap of the ping-pong schedule and adds the DDR traffic at the effective bandwidth of the board and an assumed burst latency per transfer phase.

Table 12: Equivalence per stage on 20 images per network (`results/equivalence.md`): gaps in LSB of the layer output scale, then the level of the later stages. The co-simulation and board stages are pending.

Network	Float → fake-quant, mean	max	Fake-quant → integer, per layer	end to end	Integer → golden	Golden → C-sim
`tiny-yolov2-voc`	0.50	126	1	14	identical	identical
`tiny-yolov3-coco`	0.78	93	1	17	identical	identical
`tiny-yolov4-coco`	0.91	102	1	26	identical	identical

For Tiny-YOLOv2, it gives $83.4^{\text{proj.}}$ ms for the accelerator, against $37.2^{\text{proj.}}$ ms for the overlapped cycle model. The post-processing is the data-dependent part: bounded by its 256 slots and by the candidates that can pass the threshold, it brings the frame bound to $92.4^{\text{proj.}}$ ms, hence a watchdog of $116^{\text{proj.}}$ ms. Adversarial inputs in C simulation stay under the bound. On the sigmoid heads of Tiny-YOLOv3 and YOLOv4-tiny, the uncapped NMS dominates the worst case: a demonstrable bound there needs a cap on the candidates. The burst latency and the end-to-end latency on the board remain to be measured.

12 Beyond VOC

Milestones M11, M15 and M16 take Tiny-YOLOv3 to other datasets: VisDrone (aerial views), KITTI (driving), FLIR (thermal), ExDark (low light) and CrowdHuman (crowds). Each dataset has generated notebooks for statistics, inference and a sweep of batch size × training subset, run on a Colab GPU from COCO weights. The first sweeps use short runs of 600 iterations and score them on 50 images (tier R): these scores *rank* runs and are not publishable results. On VOC itself, the published Tiny-YOLOv2 weights score higher on these 50 images than on the full split, which gives the order of magnitude of the bias.

Table 13: Best run of the first sweep of each dataset (`docs/tasks/resultats-balayages.md`).

Dataset	Training images	Best run	Batch	Training subset	Metric	Score (50 images)	Status
VOC	16,551	`b32-sall`	32	all	mAP, 11 points	36.27	tier-R
VisDrone	6,471	`b32-sall`	32	all	mAP, 11 points	2.24	tier-R
KITTI	5,985	`b32-sall`	32	all	mAP, 11 points	2.38	tier-R
FLIR	10,742	`b8-sall`	8	all	AP@[.5:.95] (COCO)	0.70	tier-R
ExDark	3,000	`b32-sall`	32	all	mAP, 11 points	13.29	tier-R
CrowdHuman	15,000	`b16-sall`	16	all	mAP, 11 points	35.87	tier-R

The scores of Table 13 show that none of these runs has converged. On VisDrone, objects are tiny at 416 pixels, many targets collide in the same cell, and the 50-image ranking does not hold on the full split; the M15 campaign replaces the sweep by longer runs, matching resize between training and evaluation, and tiles. FLIR uses the COCO metric, which does not compare with the 11-point AP of the others. The sweep figures (`balayage`, `balayages`) are not yet published in `results/figures/` and will be cited here once they are. Figure 30 shows detections of the three VOC stages.

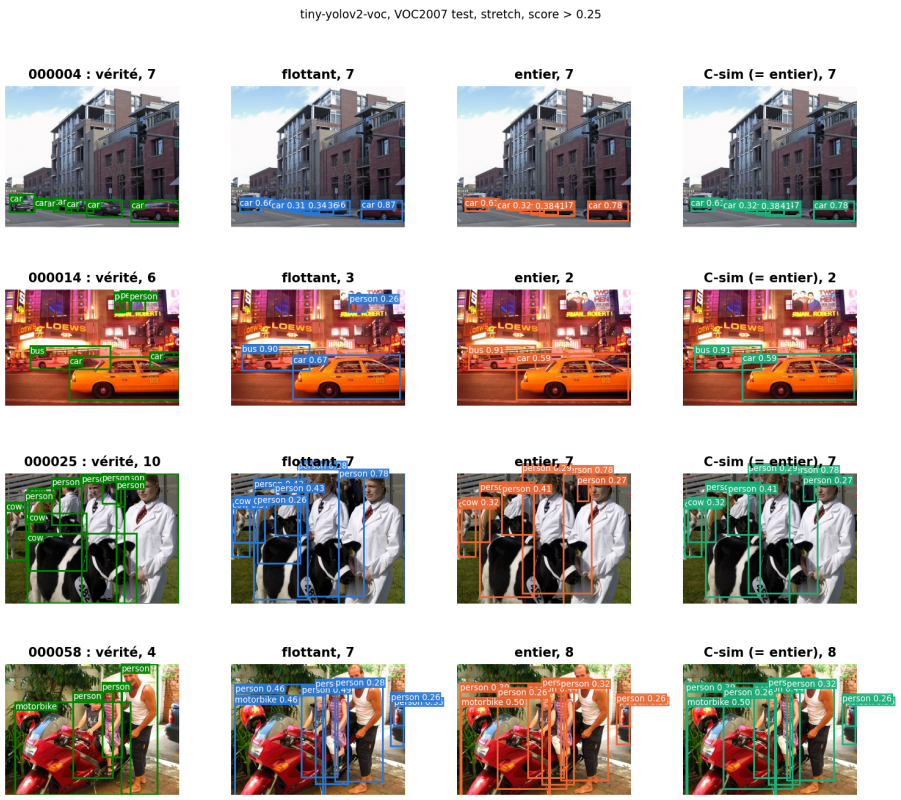


Figure 30: Vérité terrain, flottant, entier et C-sim sur quatre images de VOC2007 test : les colonnes entier et C-sim sont identiques. (`detections`)

13 Limitations and future work

The main limitation is that no number of this article has been measured on the board. The KV260, Vivado and Vitis are not available yet (milestones M6 to M8): performance is a cycle-model projection, resources are estimates, and the FPGA mAP is a C simulation. The keys still in that state are listed below; milestone task T17.19 switches them to measurements and rewrites the abstract, Section 8 and this section.

- **projection** (17): `pack.table`, `pack.v2.tm64.ms`, `pack.v2.tn32.ms`, `perf.cascade`, `perf.kv260`, `perf.kv260.bound.ms`, `perf.kv260.m6.ms`, `perf.kv260.roofline.ms`, `perf.kv260.v2.eff`, `perf.kv260.v2.fps`, `perf.kv260.v2.gops`, `perf.kv260.v2.ms`, `perf.kv260.v3.ms`, `perf.kv260.v4.ms`, `wcet.v2.acc.ms`, `wcet.v2.frame.ms`, `wcet.v2.watchdog.ms`
- **estimate** (5): `stream.w4.fps`, `stream.w8.dsp`, `stream.w8.fps`, `stream.w8.latency`, `winograd.table`
- **tier-R** (9): `balayage.crowdhuman.meilleur.map`, `balayage.exdark.meilleur.map`, `balayage.flir.meilleur.map`, `balayage.kitti.meilleur.map`, `balayage.visdrone.meilleur.map`, `balayage.voc.meilleur.map`, `balayages.meilleurs`, `quant.sensitive.gap`, `quant.sensitive.layer`
- **csim** (6): `abft.hls.layers`, `abft.hls.outputs`, `equiv.table`, `map.voc.csim`, `map.voc.identical`, `verif.hwpp.cases`

Other limits: the power-of-two ADMM did not converge and the 4-bit QAT ran for a fraction of an epoch (Section 9); the training from scratch in NumPy (fine-tuning on VOC, M2) is not finished, so the reference mAP uses the published Darknet weights; the datasets beyond VOC have only been explored at tier R (Section 12). YOLOv4-tiny still needs its full C-simulation pass on COCO and quantization-aware training to come within one point; YOLOv7-tiny has no accuracy yet, and YOLOv5n and YOLOv8n are read without PyTorch, but their decoding, targets, losses and training

in NumPy are still to write (M33 to M35). The fault campaign has only run as a smoke test, and the ABFT lane of the kernel is not synthesized. The equivalence evidence and the latency bound of Section 11 stop at the C simulation: the co-simulation and board rows, the measured burst latency and the latency distribution on the board are pending. The person-detection safety metrics, the runtime monitors and the assurance case (M27, M29, M31) wait for the CrowdHuman data on the PC; the explanation of the decisions (M36) is planned. The long runs still open are listed with their durations in `docs/tasks/long-runs.md`. Table 14 gives the progress by milestone.

14 Reproducibility

Every number of this article is regenerated from the repository: `make article` reads the results (`collect`) into `docs/article/chiffres.json`, which records for each key its value, status, source file, command and the revision where it last changed, then rewrites `docs/article/generated.tex` (`render`), the only file this article takes its numbers from. `python -m tools.article -check` runs in `make ci`, and `make article-pdf` compiles this document. The main commands are:

Stage	Command
tests (Python, C++)	`make test`, `make ci`
float and integer mAP	`make eval-float`, `make eval-int`
export and golden model	`make export`, `make golden-check`
HLS kernel in C simulation	`make csim-gcc`, `make hls-cycles`
ARM driver on PC	`make sw-sim`
mAP at the three stages	`make m8-inputs m8-int bench-sim map-stades`
COCO, YOLOv4-tiny	`tools/m11.sh coco-v4`
faults, OOD monitor	`tools/m25.sh`, `tools/m25.sh ood`
equivalence, proofs, traceability	`make equivalence`, `make prove-kernels`, `make traceability`
latency bound	`python tools/perf_model.py -wcet`
performance projection	`make perf-model bench-report`
figures, notebooks	`make figures`, `make notebooks`
this article	`make article`, `make article-pdf`

Each evaluation has a test profile (M12): tier R on a few images for development, tier M on a mid-size subset, tier N on the full split; only tier N numbers go to `results/`. The notebooks of M14 run the same tools on Colab and are versioned with their outputs (Figures 31 and 32). This document was rendered at rev. `9fd9562-dirty`, 2026-10-22.

References

Papers of the article base, cited with the chunk that supports the statement (`[n, #chunk]`), checkable with `bin/pdb verify`, a tool of the base outside this repository. Statements no paper of the base supports are marked "outside the base": the hardware handling of the 2×2 stride-1 max pooling, the Darknet `.cfg` files (exact channels, Tiny-YOLOv3 anchors), the integer requantization chain and the BN derivatives. YOLOv3 is a technical report that was never published at a conference, but it is the de facto reference; preprints have not been peer reviewed.

[1] Fata and Elmannai. Low-cost FPGA-enhanced CNN accelerator for real-time YOLO object detection and classification. *IEEE Access*, 14, 2026. doi: 10.1109/ACCESS.2026.3669451.

[2] Redmon et al. You only look once: Unified, real-time object detection. In *Proceedings of CVPR*, 2016. doi: 10.1109/CVPR.2016.91.

Table 14: Progress by milestone (`docs/tasks/README.md`).

Milestone	Tasks	Done	Progress
M0	6	6	100 %
M1	9	8	89 %
M2	9	8	89 %
M3	4	4	100 %
M4	7	7	100 %
M5	6	6	100 %
M6	6	1	17 %
M7	4	0	0 %
M8	3	0	0 %
M9	1	0	0 %
M9.1	3	2	67 %
M9.2	4	3	75 %
M9.3	6	5	83 %
M9.4	3	3	100 %
M10	14	0	0 %
M12	11	0	0 %
M11	8	1	12 %
M13	60	60	100 %
M14	12	8	67 %
M15	16	1	6 %
M16	20	18	90 %
M17	21	17	81 %
M18	11	9	82 %
M19	12	5	42 %
M20	10	9	90 %
M21	12	6	50 %
M22	12	11	92 %
M23	6	6	100 %
M24	7	5	71 %
M25	9	5	56 %
M26	6	2	33 %
M27	7	0	0 %
M28	7	6	86 %
M29	6	1	17 %
M30	4	1	25 %
M31	6	0	0 %
M32	14	3	21 %
M33	9	4	44 %
M34	6	0	0 %
M35	8	0	0 %
M36	12	0	0 %
Total	**397**	**231**	**58 %**

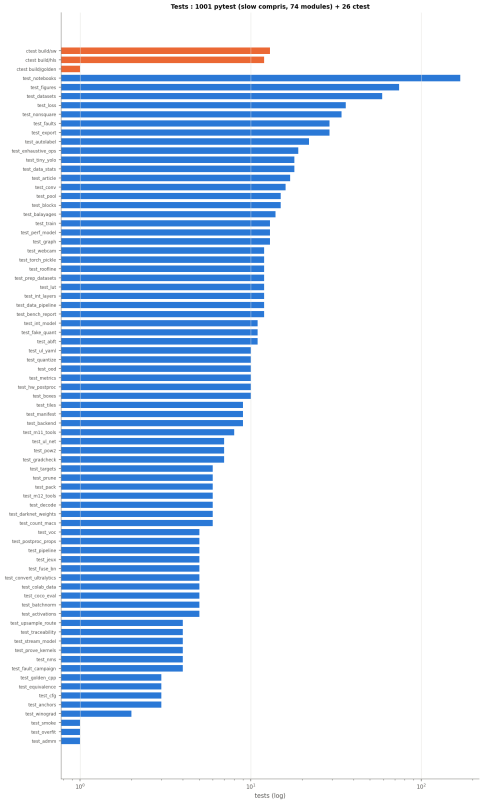


Figure 31: Nombre de tests par module Python (pytest, slow compris) et par build C++ (ctest), et leur durée quand `make test-durations` a été lancé. (`tests`)

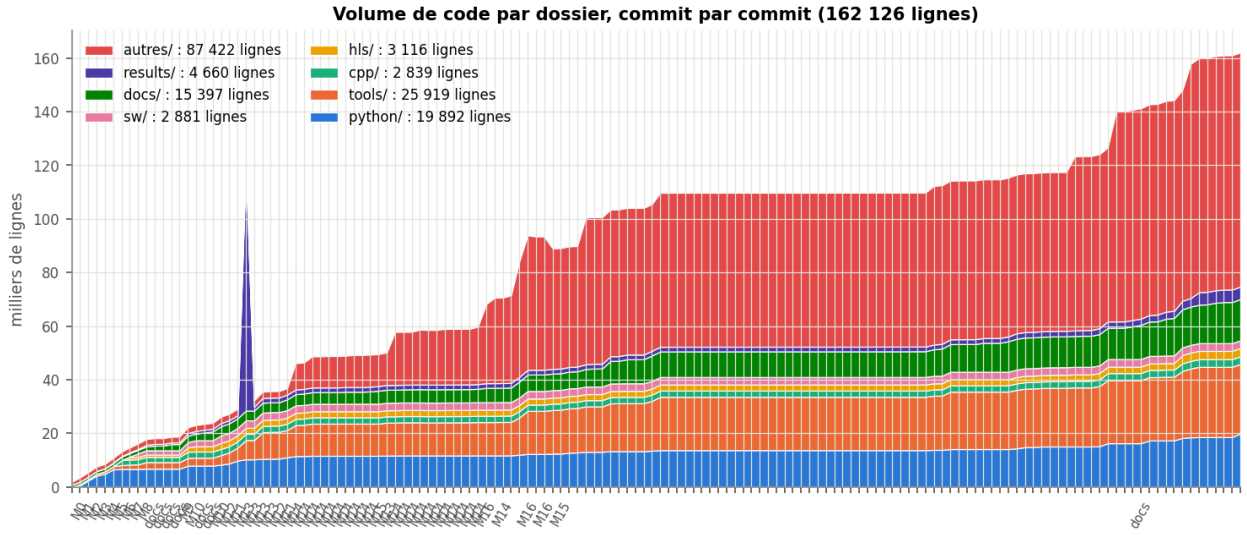


Figure 32: Lignes par dossier (python, tools, cpp, hls, sw, docs, results) commit par commit. (`code`)

[3] Redmon and Farhadi. YOLO9000: Better, faster, stronger. In *Proceedings of CVPR*, 2017. URL https://arxiv.org/abs/1612.08242.

[4] Zhai et al. FPGA-based vehicle detection and tracking accelerator. *Sensors*, 23(4), 2023. doi: 10.3390/s23042208.

[5] Venieris et al. Toolflows for mapping convolutional neural networks on FPGAs: A survey and future directions. *ACM Computing Surveys*, 51(3), 2018. doi: 10.1145/3186332.

[6] Montgomerie-Corcoran et al. SATAY: A streaming architecture toolflow for accelerating YOLO models on FPGA devices. In *Proceedings of ICFPT*, 2023. doi: 10.1109/ICFPT59805.2023.00025.

[7] Zhang et al. Optimizing FPGA-based accelerator design for deep convolutional neural networks. In *Proceedings of FPGA '15 (ACM/SIGDA)*, 2015. doi: 10.1145/2684746.2689060.

[8] Zhang et al. End-to-end acceleration of the YOLO object detection framework on FPGA-only devices. *Neural Computing and Applications*, 36(3), 2024. doi: 10.1007/s00521-023-09078-8.

[9] Ding et al. REQ-YOLO: A resource-aware, efficient quantization framework for object detection on FPGAs. In *Proceedings of FPGA '19 (ACM/SIGDA)*, 2019. doi: 10.1145/3289602.3293904.

[10] Kim and Kim. Design and implementation of a YOLOv2 accelerator on a Zynq-7000 FPGA. *Sensors*, 2025. doi: 10.3390/s25206359.

[11] Redmon and Farhadi. YOLOv3: An incremental improvement, 2018. URL https://arxiv.org/abs/1804.02767. arXiv preprint.

[12] Dong et al. CoDeNet: Efficient deployment of input-adaptive object detection on embedded FPGAs. In *Proceedings of FPGA '21*, 2021. doi: 10.5281/zenodo.4341394.

[13] Tang et al. EF-Train: Enable efficient on-device CNN training on FPGA through data reshaping for online adaptation or personalization, 2022. URL https://arxiv.org/abs/2202.10935. arXiv preprint.

[14] Karki et al. No attention, no problem: DPU-aware attention approximation in modern YOLO on FPGA, 2026. URL https://arxiv.org/abs/2607.13106. arXiv preprint.

[15] Yan et al. An FPGA-based YOLOv5 accelerator for real-time industrial vision applications. *Micromachines*, 15(9), 2024. doi: 10.3390/mi15091164.

A Version log

Written by hand, one entry per update of the article (procedure "Mise à jour à chaque jalon" of M17).

Version	Date	Changes
v0	2026-10-07	Infrastructure (`tools/article.py`, `make article`, `-check` in CI) and first draft of every section, in English (M19 rule). Board measurements pending.
v1	2026-10-07	Source moved from Markdown to LaTeX (`article.tex`, `generated.tex`, `refs.bib`); PDF compiled by xelatex.
v2	2026-10-20	Two MACs per DSP and the Winograd study (M23), YOLOv4-tiny on COCO (M24), new section on dependability (M25), YOLOv7-tiny in progress (M26).
v3	2026-10-22	New section on safety evidence: equivalence matrix, operator tests, SMT proof and traceability (M28), static latency bound (M30); ABFT lane and buffer faults in the HLS kernel (M25); YOLOv5n read without PyTorch (M33).